

MBA EN **INGENIERIA**
DE SOFTWARE

PATRONES DE
PROYECTOS (*DESIGN*
PATTERNS) II

Prof.^a Dr.^a Rosana T. Vaccare Braga

MBAUSP
ESALQ

La responsabilidad por la idoneidad, originalidad y licitud de los contenidos didácticos presentados es del profesor.

Prohibida la reproducción, total o parcial, sin autorización.

Ley nº 9610/98

En la clase anterior ...

- Reutilización de *software* y sus beneficios
- Patrones y su origen
- Elementos de un patrón, tipos de patrón
- Patrones de proyectos: definiciones, ventajas, categorías, formatos de escritura
- Patrones GoF de la categoría “de creación” y 2 patrones de la categoría “estructural”

		Propósito		
		De creación	Estructurales	Conductual
Alcance	Clase	Método de fábrica	Adaptador	Intérprete Método hoja de respuestas
	Objeto	• Fábrica Abstracta • Constructor • Prototipo • Objeto Unitario (Singleton)	• Adaptador • Puente • Compuesto • Decorador • Fachada • Peso pluma • Proxy (Procurador)	• Cadena de Responsabilidad • Comando • Iterador • Mediador • Memento • Observador • Estado • Estrategia • Visitador

En la clase de hoy

- Otros patrones de la categoría “estructural”;
- Patrones de la categoría “comportamental”;
- Relación entre los patrones de proyectos;
- Conclusiones
- Discusión: Impacto de la Inteligencia Artificial en los patrones de proyecto

		Propósito		
		De creación	Estructurales	Conductual
Alcance	Clase	Método de fábrica	Adaptador	Intérprete Método hoja de respuesta
	Objeto	• Fábrica Abstracta • Constructor • Prototipo • Objeto Unitario (Singleton)	• Adaptador • Puente • Compuesto • Decorador • Fachada • Peso pluma • Proxy (Procurador)	• Cadena de Responsabilidad • Comando • Iterador • Mediador • Memento • Observador • Estado • Estrategia • Visitador

Clase de hoy:

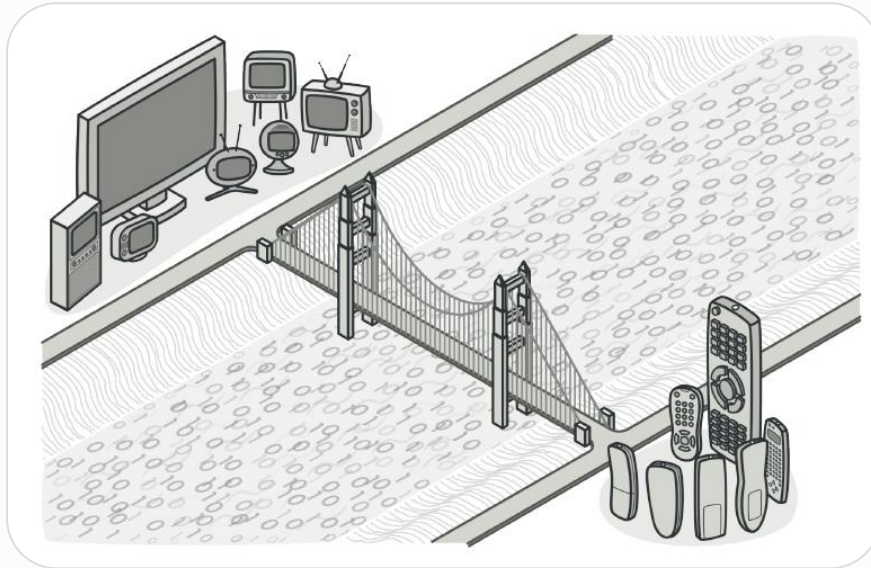
		Propósito		
		De creación	Estructurales	Conductual
Alcance	Clase	Método de fábrica	Adaptador	Intérprete Método plantilla
	Objeto	- Fábrica Abstracta - Constructor - Prototipo - Objeto Único (<i>Singleton</i>)	- Adaptador - Puente - Compuesto - Decorador - Fachada - Peso pluma <i>Proxy</i>	- Cadena de Responsabilidad - Comando - Iterador - Mediador - Memento - Observador - Estado - Estrategia - Visitante

PATRONES de la categoría “estructural”

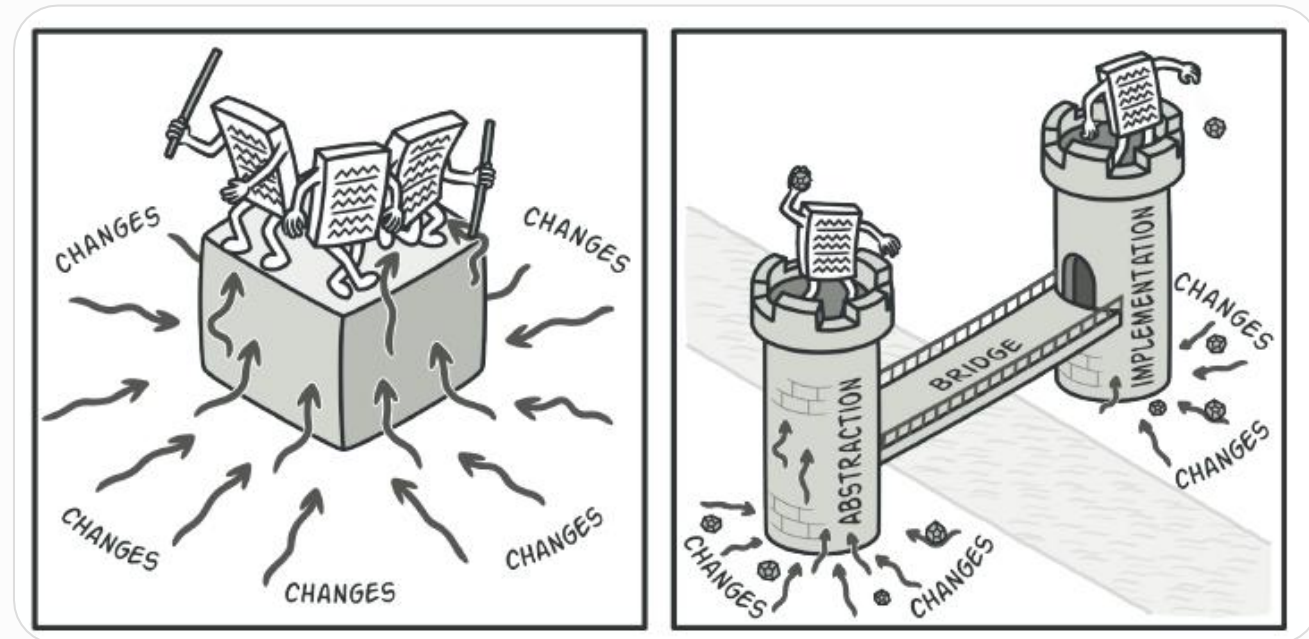
Se preocupan por la forma en que ocurren las composiciones de clases y objetos en estructuras mayores

Retomando de la clase anterior: Patrón de Proyecto Bridge

- **Clasificación:** Patrón estructural/Ámbito de objetos
- **Intención:** Desacoplar una abstracción de su implementación para que ambas puedan variar independientemente.

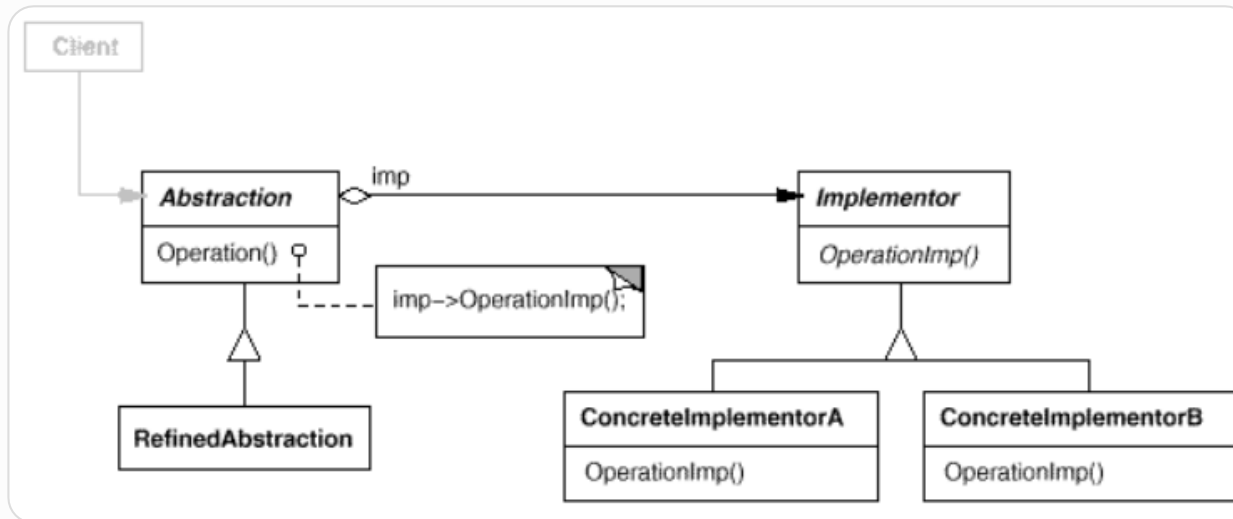


(Shvets, Alexander, 2018)

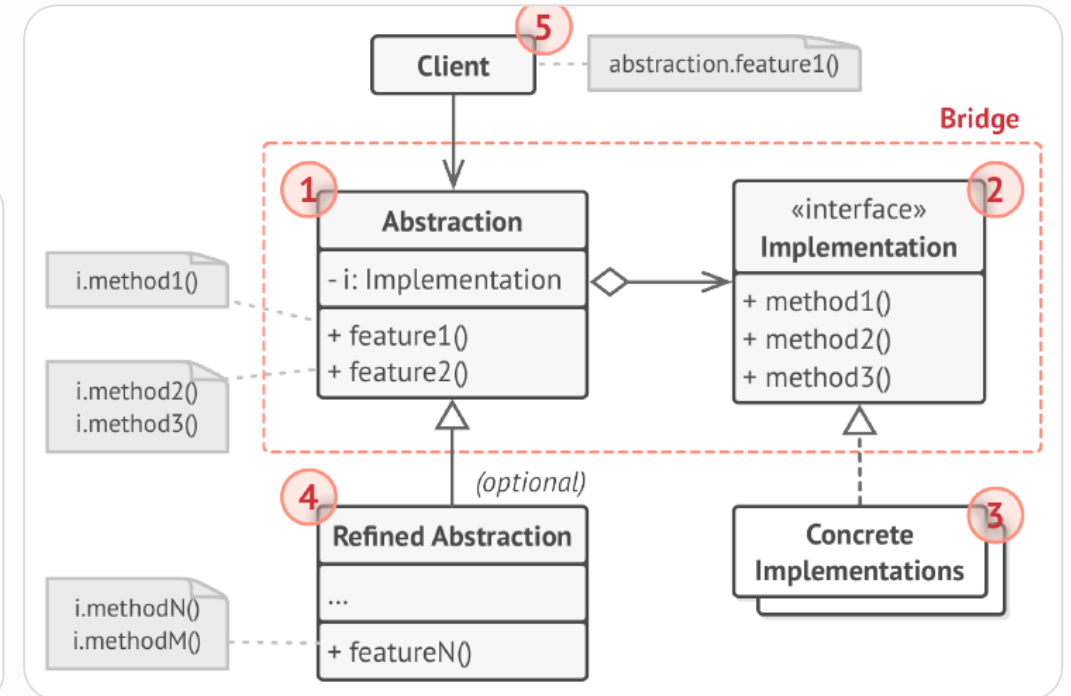


Patrón de Proyecto Puento

- Estructura:



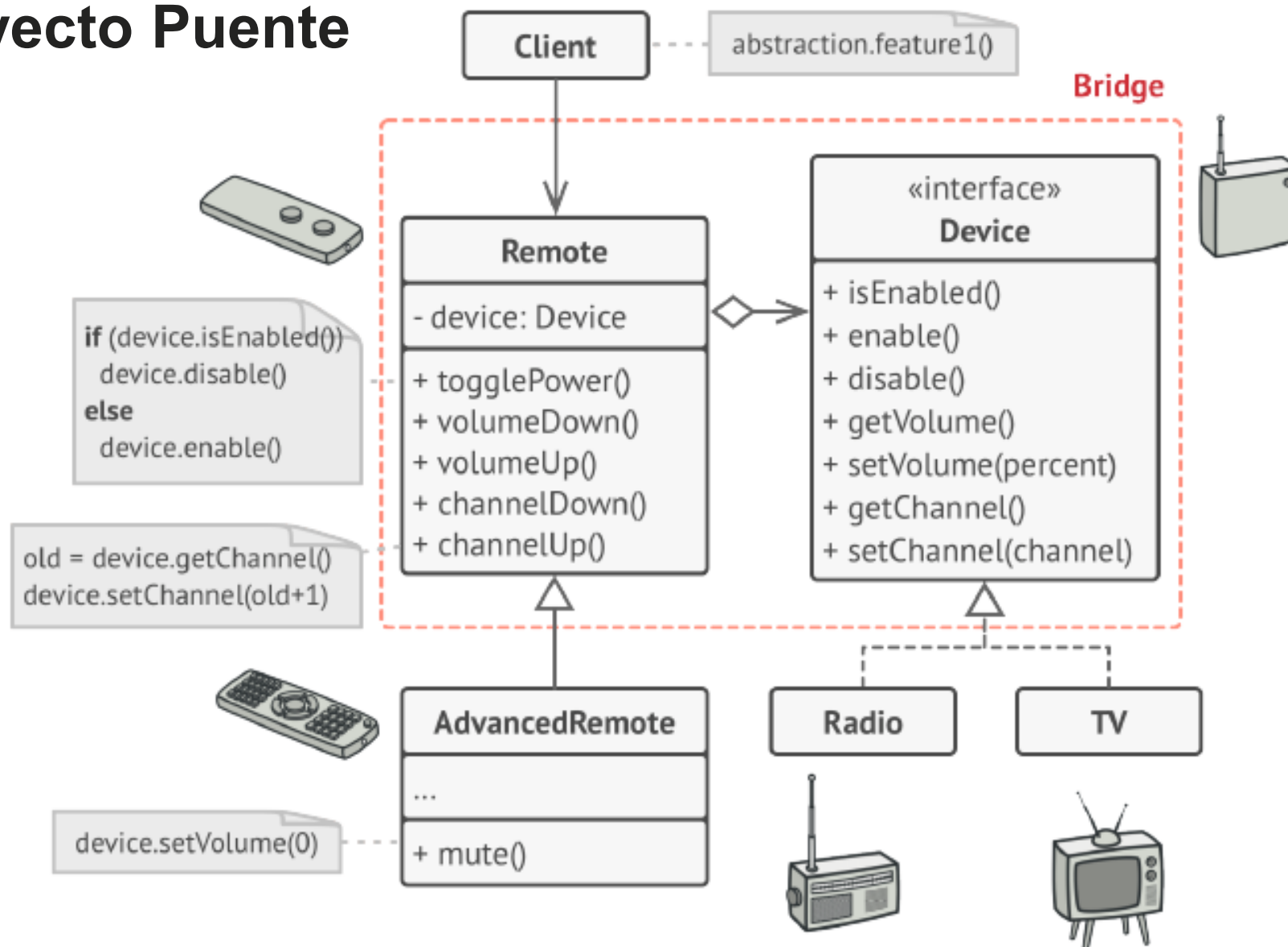
** Imagen capturada en el libro versión html publicado en CD (Gamma et al 1995)



(Shvets, Alexander, 2018)

Patrón de Proyecto Puente

- Ejemplo:



(Shvets, Alexander, 2018)

Patrón de Proyecto Puento

- Ejemplo:

```
23
24 // You can extend classes from the abstraction hierarchy
25 // independently from device classes.
26 class AdvancedRemoteControl extends RemoteControl is
27     method mute() is
28         device.setVolume(0)
29
```

```
1 // The "abstraction" defines the interface for the "control"
2 // part of the two class hierarchies. It maintains a reference
3 // to an object of the "implementation" hierarchy and delegates
4 // all of the real work to this object.
5 class RemoteControl is
6     protected field device: Device
7     constructor RemoteControl(device: Device) is
8         this.device = device
9     method togglePower() is
10         if (device.isEnabled()) then
11             device.disable()
12         else
13             device.enable()
14     method volumeDown() is
15         device.setVolume(device.getVolume() - 10)
16     method volumeUp() is
17         device.setVolume(device.getVolume() + 10)
18     method channelDown() is
19         device.setChannel(device.getChannel() - 1)
20     method channelUp() is
21         device.setChannel(device.getChannel() + 1)
22
```

(Shvets, Alexander, 2018)

Patrón de Proyecto Puento

- Ejemplo:

```
37 interface Device is
38     method isEnabled()
39     method enable()
40     method disable()
41     method getVolume()
42     method setVolume(percent)
43     method getChannel()
44     method setChannel(channel)
45
46
47 // All devices follow the same interface.
48 class Tv implements Device is
49     // ...
50
51 class Radio implements Device is
52     // ...
```

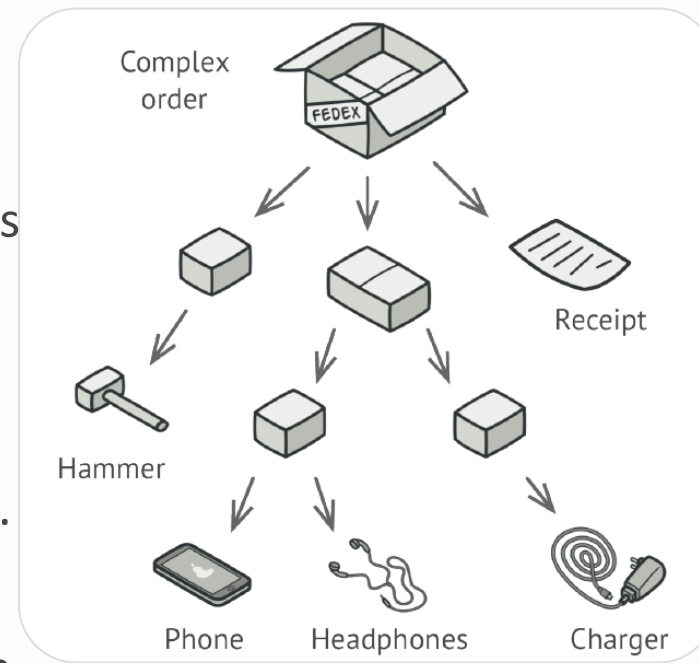
Patrón de Proyecto Puento

- Ejemplo:

```
55  // Somewhere in client code.  
56  tv = new Tv()  
57  remote = new RemoteControl(tv)  
58  remote.togglePower()  
59  
60  radio = new Radio()  
61  remote = new AdvancedRemoteControl(radio)
```

Patrón de Proyecto Compuesto (del inglés *Composite*)

- **Clasificación:** Patrón de Creación/Ámbito de objetos
- **Intención:** Componer objetos en estructuras de árbol para representar jerarquías todo-parte. El Compuesto permite que el cliente trate objetos individuales y composiciones de objetos uniformemente.
- **Motivación:** En aplicaciones gráficas, se pueden construir diagramas complejos a partir de componentes simples. Se pueden agrupar componentes para crear otros componentes mayores, en varios niveles.
 - El problema es que el código que usa estas clases debe tratar de manera diferente los objetos primitivos y los objetos que contienen otros objetos, lo que hace que el código sea más complejo.

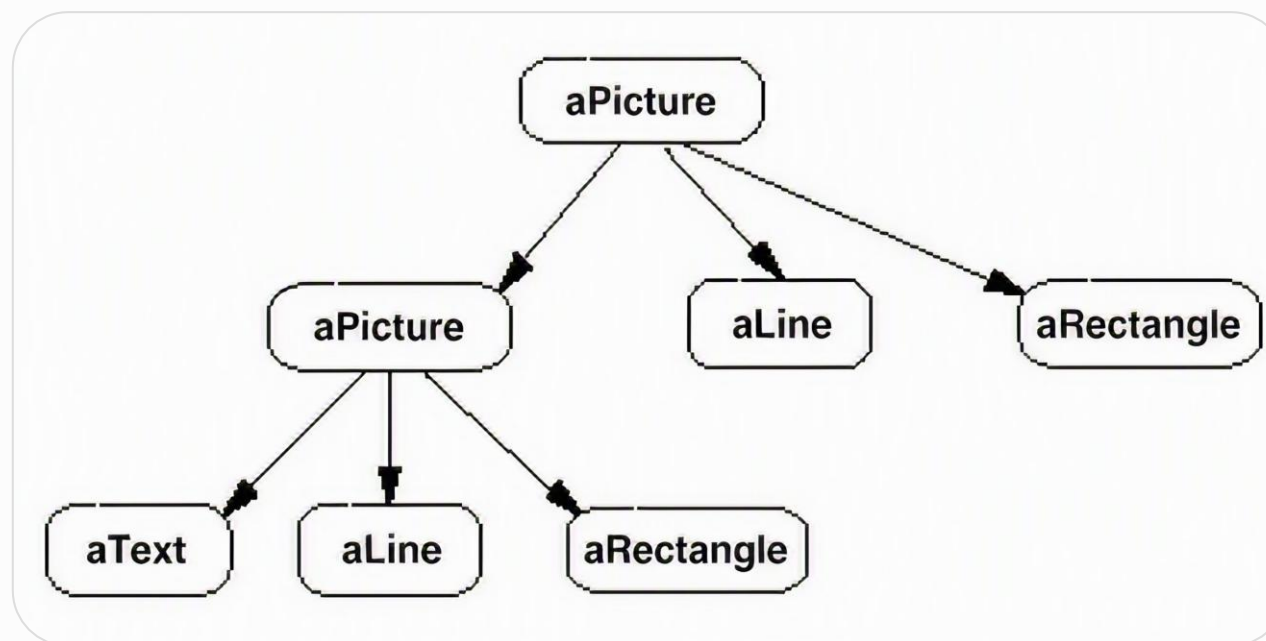


(Shvets, Alexander, 2018)

Encuesta 6: Ejemplos de Compuesto



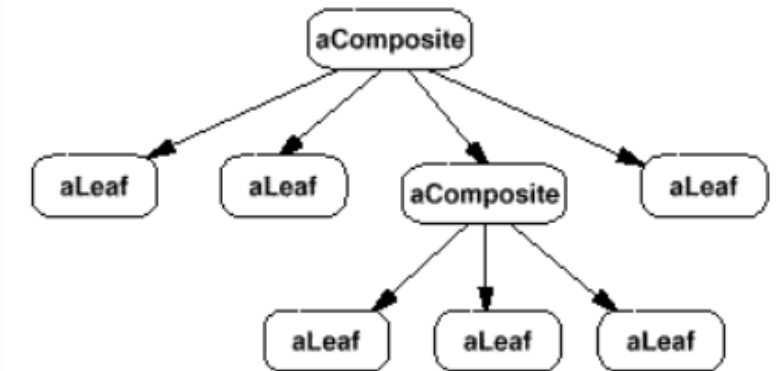
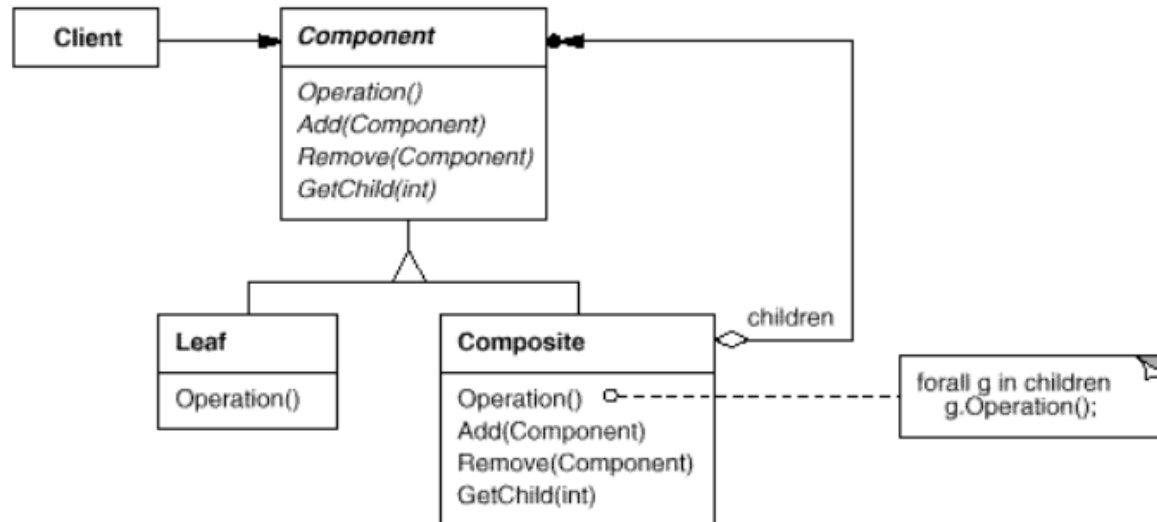
Otro ejemplo del Patrón de Proyecto Compuesto



* Imagen capturada del libro versión html publicado en CD (Gamma et al 1995)

Patrón de Proyecto Compuesto

- Aplicabilidad
- Use el patrón Compuesto cuando:
 - quiera representar jerarquías todo-parte.
 - quiera que los clientes sean capaces de ignorar la diferencia entre composiciones de objetos y objetos individuales. Los clientes tratarán todos los objetos en la estructura compuesta uniformemente.
- Estructura:



Imágenes capturadas en el libro versión html publicado en CD (Gamma et al 1995)

Patrón de Proyecto Compuesto

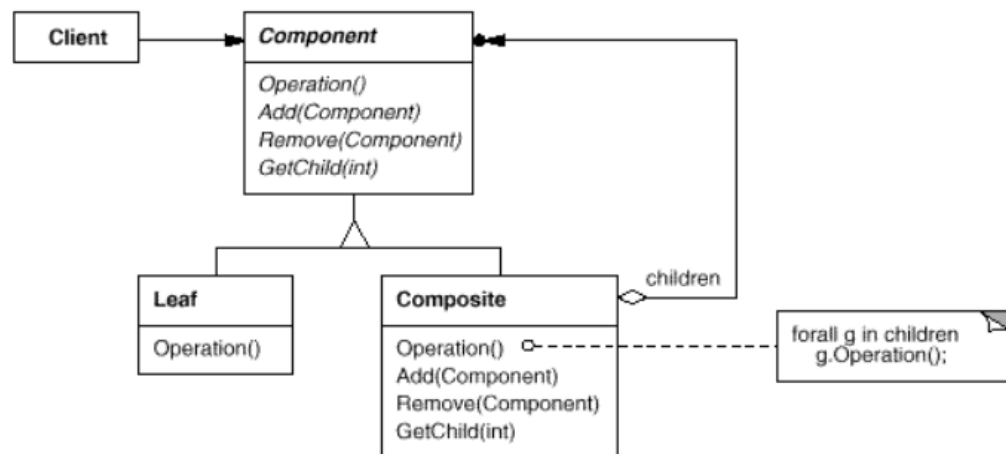
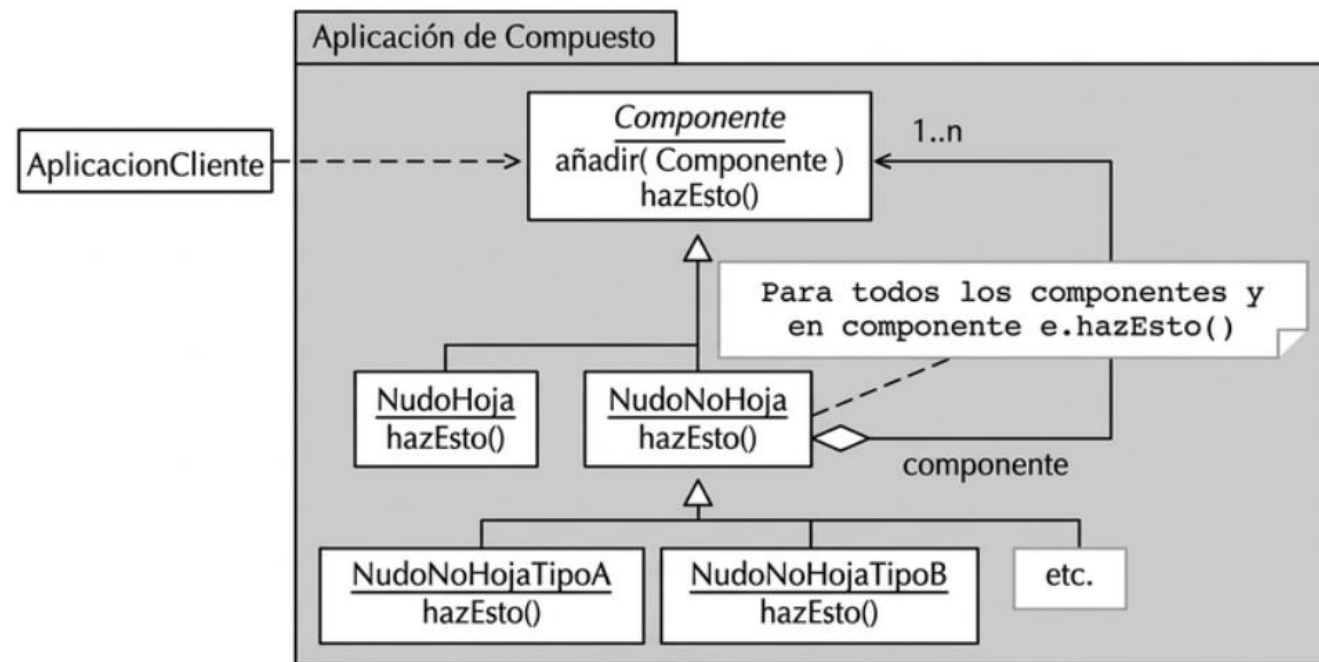


Imagen capturada en el libro versión html publicado en CD (Gamma et al 1995)



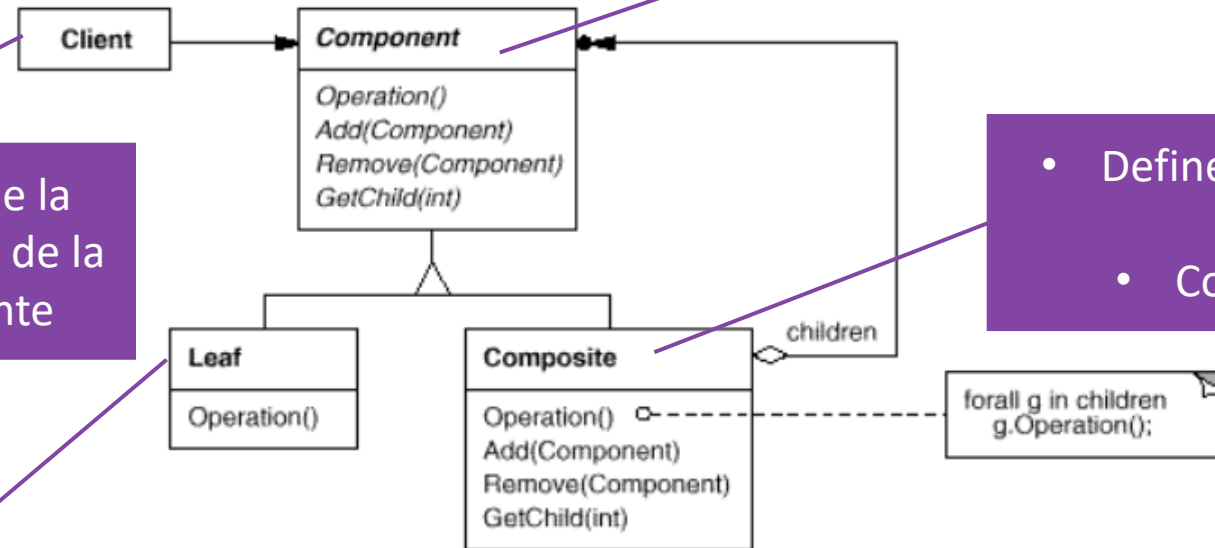
(Braude, Eric, 2005)

Patrón de Proyecto Compuesto

- Participantes:

- Declara interfaz para objetos de la composición y para gestionar componentes hijo
- El comportamiento *default* permanece aquí, cuando corresponda

- Manipula objetos de la composición a través de la interfaz Componente

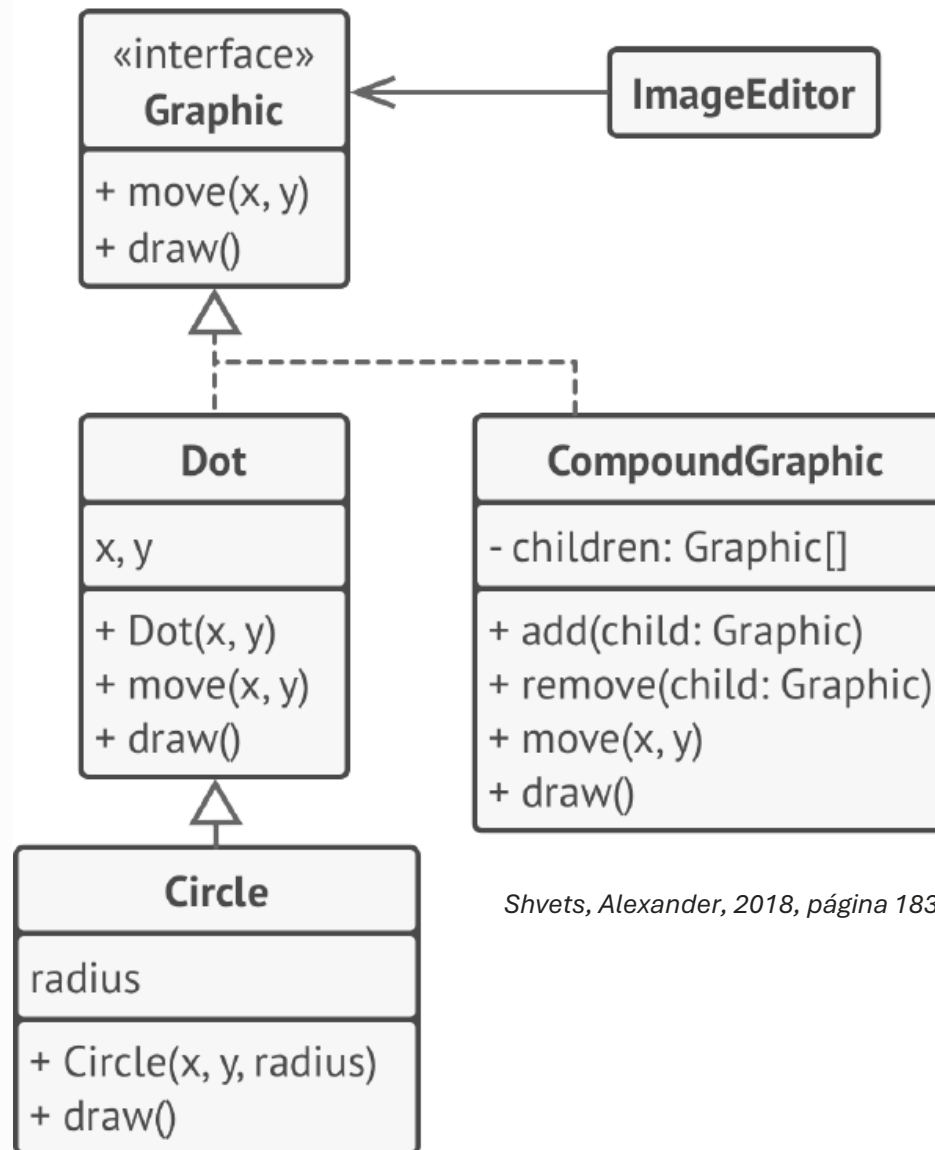


- Define comportamiento para objetos que tienen hijos
- Conoce los componentes hijo

- Representa objetos hoja (que no tienen hijos)
- Define comportamiento para objetos primitivos en la composición

** Imagen capturada en el libro versión html publicado en CD (Gamma et al 1995)

Ejemplo de código del Patrón de Proyecto Compuesto



Shvets, Alexander, 2018, página 183

Ejemplo de código del Patrón de Proyecto Compuesto

```
1  // The component interface declares common operations for both
2  // simple and complex objects of a composition.
3  interface Graphic is
4      method move(x, y)
5      method draw()
6
7  // The leaf class represents end objects of a composition. A
8  // leaf object can't have any sub-objects. Usually, it's leaf
9  // objects that do the actual work, while composite objects only
10 // delegate to their sub-components.
11 class Dot implements Graphic is
12     field x, y
13
14     constructor Dot(x, y) { ... }
15
16     method move(x, y) is
17         this.x += x, this.y += y
18
19     method draw() is
20         // Draw a dot at X and Y.
21
22 // All component classes can extend other components.
23 class Circle extends Dot is
24     field radius
25
26
```

Shvets, Alexander, 2018, página 184

Ejemplo de código del Patrón de Proyecto Compuesto

```
27     constructor Circle(x, y, radius) { ... }
28
29     method draw() is
30         // Draw a circle at X and Y with radius R.
31
32     // The composite class represents complex components that may
33     // have children. Composite objects usually delegate the actual
34     // work to their children and then "sum up" the result.
35     class CompoundGraphic implements Graphic is
36         field children: array of Graphic
37
38         // A composite object can add or remove other components
39         // (both simple or complex) to or from its child list.
40         method add(child: Graphic) is
41             // Add a child to the array of children.
42
43         method remove(child: Graphic) is
44             // Remove a child from the array of children.
45
46         method move(x, y) is
47             foreach (child in children) do
48                 child.move(x, y)
49
50         // A composite executes its primary logic in a particular
51         // way. It traverses recursively through all its children,
52         // collecting and summing up their results. Since the
53         // composite's children pass these calls to their own
54         // children and so forth, the whole object tree is traversed
55         // as a result.
56         method draw() is
57             // 1. For each child component:
58             //     - Draw the component.
```

Shvets, Alexander, 2018, página 185

Ejemplo de código del Patrón de Proyecto Compuesto

```
59      //      - Update the bounding rectangle.
60      // 2. Draw a dashed rectangle using the bounding
61      // coordinates.
62
63
64      // The client code works with all the components via their base
65      // interface. This way the client code can support simple leaf
66      // components as well as complex composites.
67      class ImageEditor is
68          method load() is
69              all = new CompoundGraphic()
70              all.add(new Dot(1, 2))
71              all.add(new Circle(5, 3, 10))
72              // ...
73
74      // Combine selected components into one complex composite
75      // component.
76      method groupSelected(components: array of Graphic) is
77          group = new CompoundGraphic()
78          group.add(components)
79          all.remove(components)
80          all.add(group)
81          // All components will be drawn.
82          all.draw()
```

Shvets, Alexander, 2018, página 186

Consecuencias de uso del Compuesto

- Los objetos primitivos pueden combinarse para formar objetos más complejos, que a su vez pueden combinarse, y así sucesivamente de forma recursiva:
 - siempre que el código del cliente espere un objeto primitivo, también puede recibir un objeto compuesto.
- Simplifica al cliente.
 - Los clientes no necesitan saber si están tratando con una hoja o un componente compuesto
 - evita tener que escribir funciones al estilo *“do case”*
- Facilita la adición de nuevos tipos de componentes: los clientes no necesitan ser modificados para nuevas clases Componente.
- Puntos negativos:
 - el diseño puede volverse muy genérico.
 - se vuelve más difícil restringir los componentes de un compuesto. Por ejemplo, si es necesario que su compuesto tenga solo ciertos componentes. No puede confiar en el sistema de tipos del lenguaje de programación para imponer estas restricciones, sino que tendrá que usar verificaciones en tiempo de ejecución.

Patrones relacionados al Compuesto

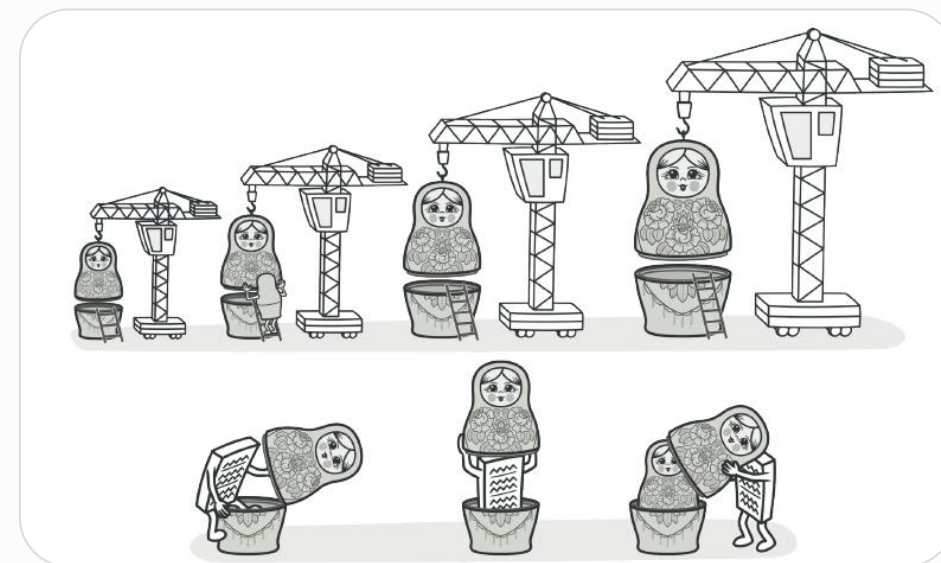
- *Chain of Responsibility*
- Decorator
- *Flyweight*
- Iterator
- Visitor

Retomando la encuesta 6

- ¿Puede dibujar un diagrama de clases aplicando el Patrón Compuesto en otro ejemplo?

Patrón de Proyecto Decorador (del inglés *Decorator*)

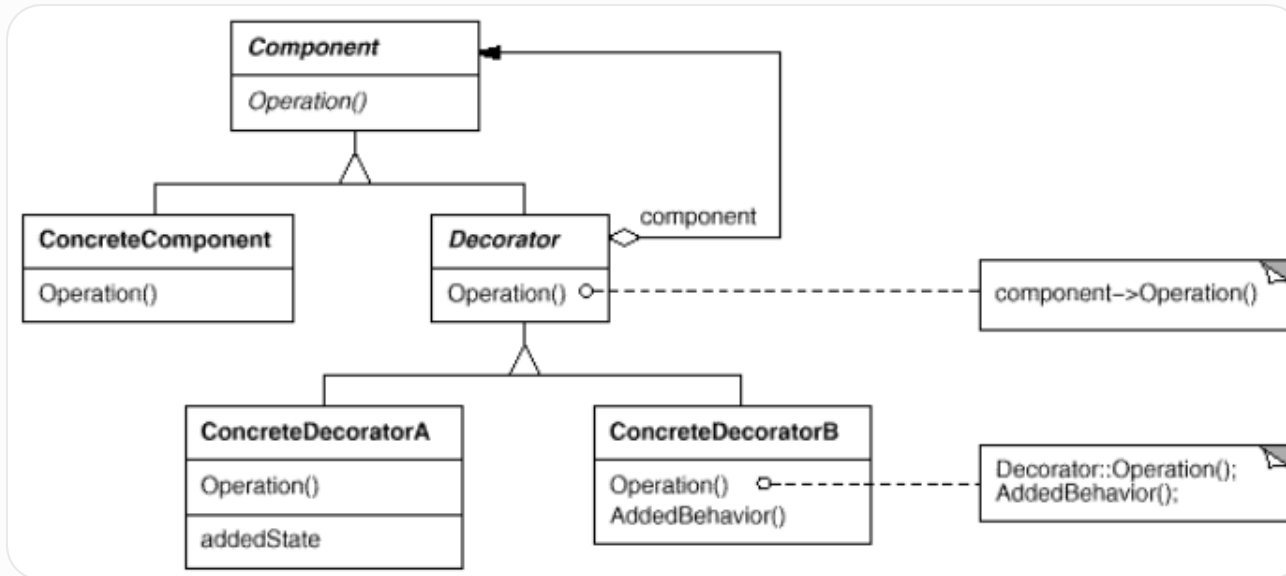
- **Clasificación:** Patrón estructural/Ámbito de objetos
- **Intención:** Asignar responsabilidades adicionales a un objeto dinámicamente. Los decoradores proporcionan una alternativa flexible a la herencia para extender la funcionalidad.
- **Aplicación:** Use Decorador para:
 - añadir responsabilidades a objetos individuales de forma dinámica y transparente, es decir, sin afectar a otros objetos.
 - Permitir que las responsabilidades puedan ser retiradas.
 - Evitar que un gran número de extensiones independientes produzca una explosión de subclases para dar soporte a cada combinación.



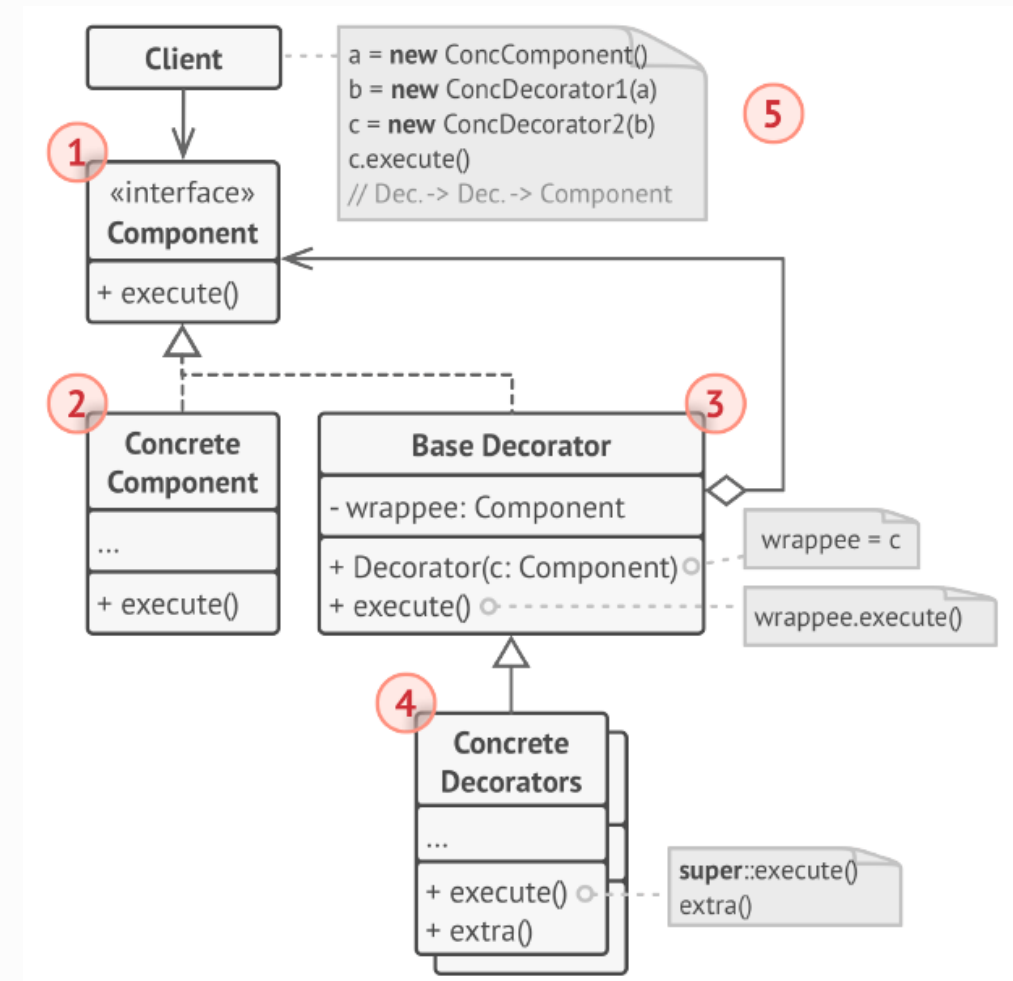
(Shvets, Alexander, 2018)

Patrón de Proyecto Decorador

- Estructura:

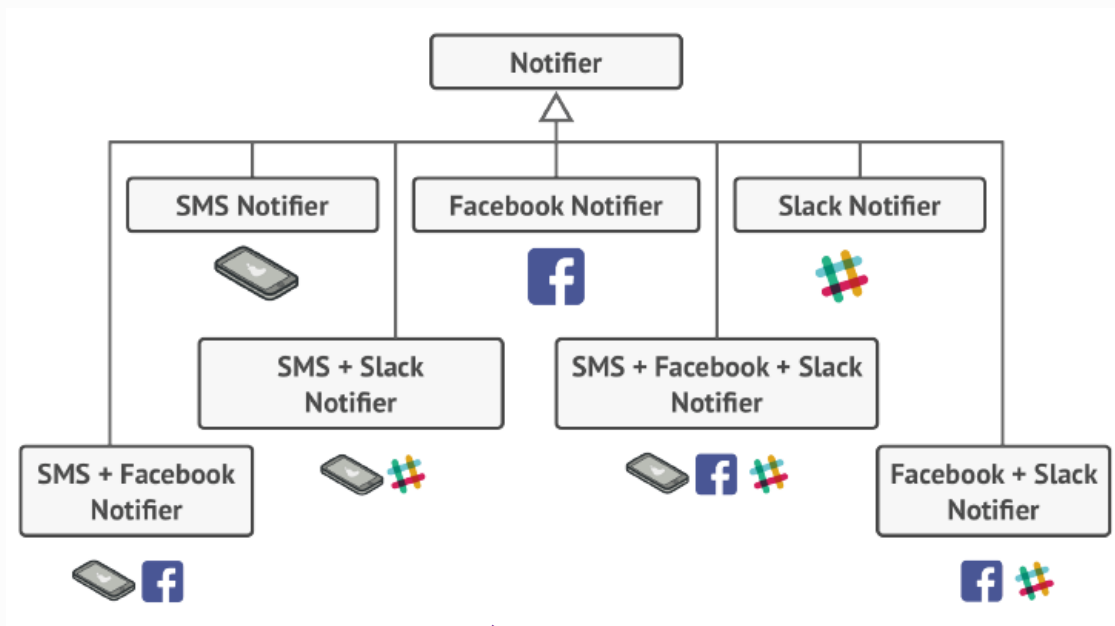


** Imagen capturada en el libro versión
html publicado en CD (Gamma et al 1995)

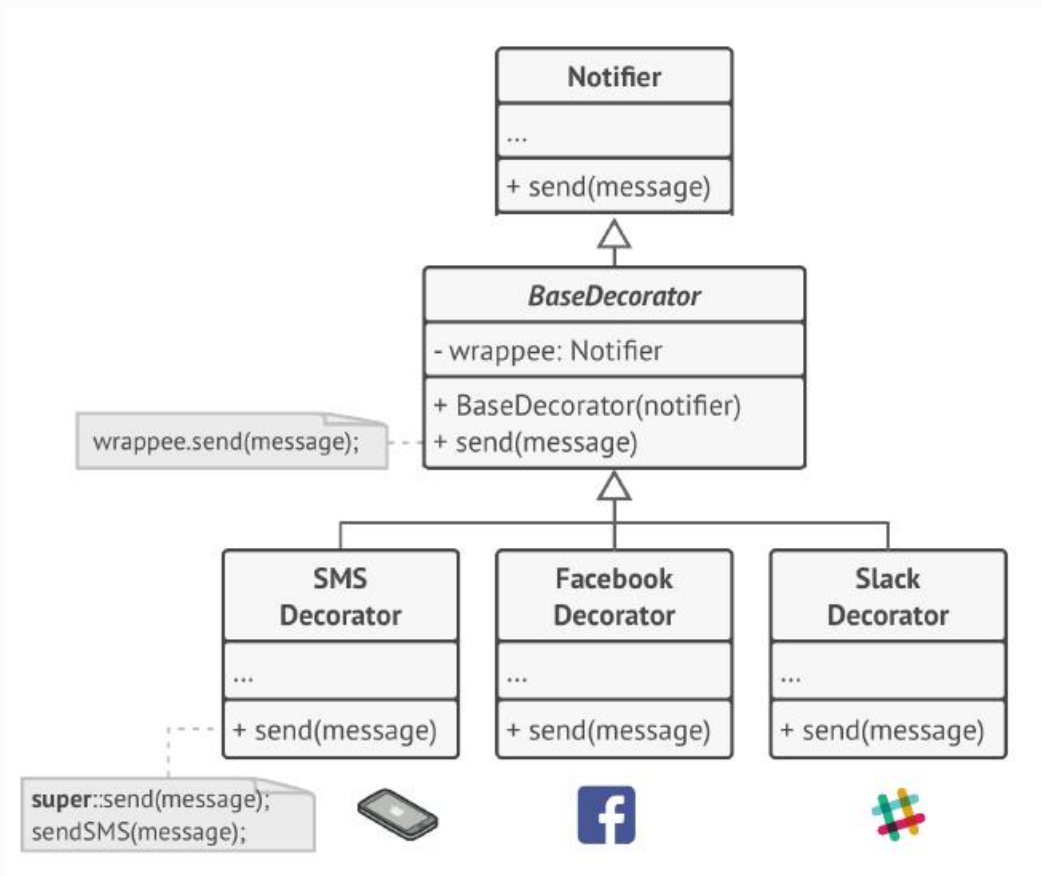


(Shvets, Alexander, 2018)

Patrón de Proyecto Decorador



- Sin usar el *Decorator*

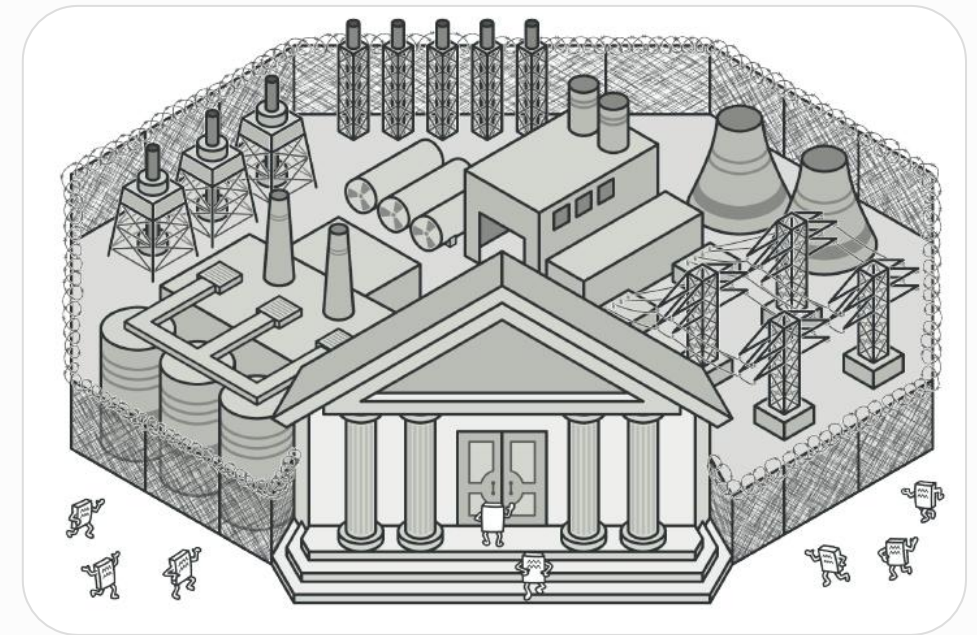


- Usando el *Decorator*

(Shvets, Alexander, 2018)

Patrón de Proyecto Fachada (del inglés Facade)

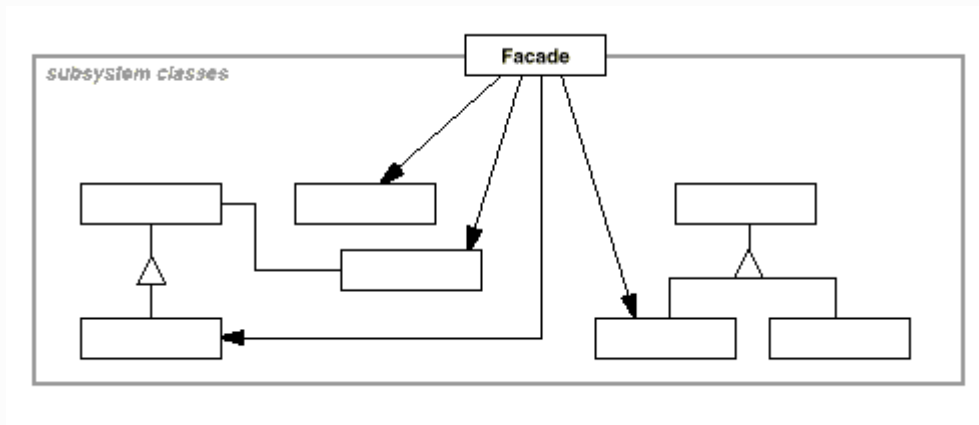
- **Clasificación:** Patrón estructural/Ámbito de objetos
- **Intención:** Proporcionar una interfaz unificada para un conjunto de interfaces en un subsistema. Las fachadas definen una interfaz de nivel superior que facilita el uso del subsistema.
- **Aplicación:** necesita integrar su aplicación o sistema con una biblioteca sofisticada que tiene decenas de características, pero solo necesita una parte de la funcionalidad



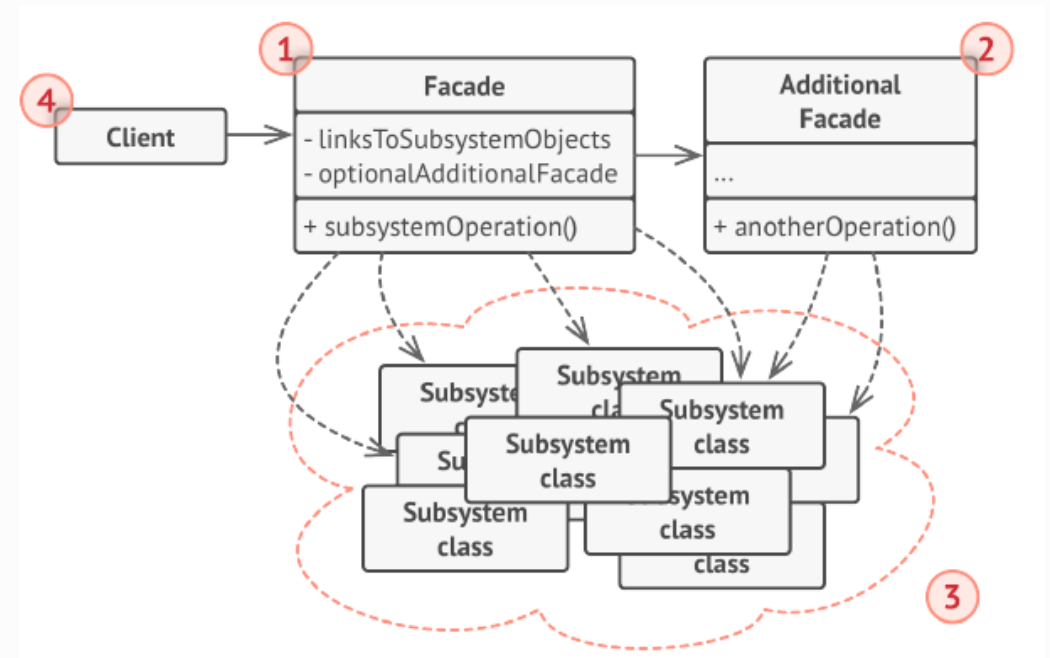
(Shvets, Alexander, 2018)

Patrón de Proyecto Fachada

- Estructura:



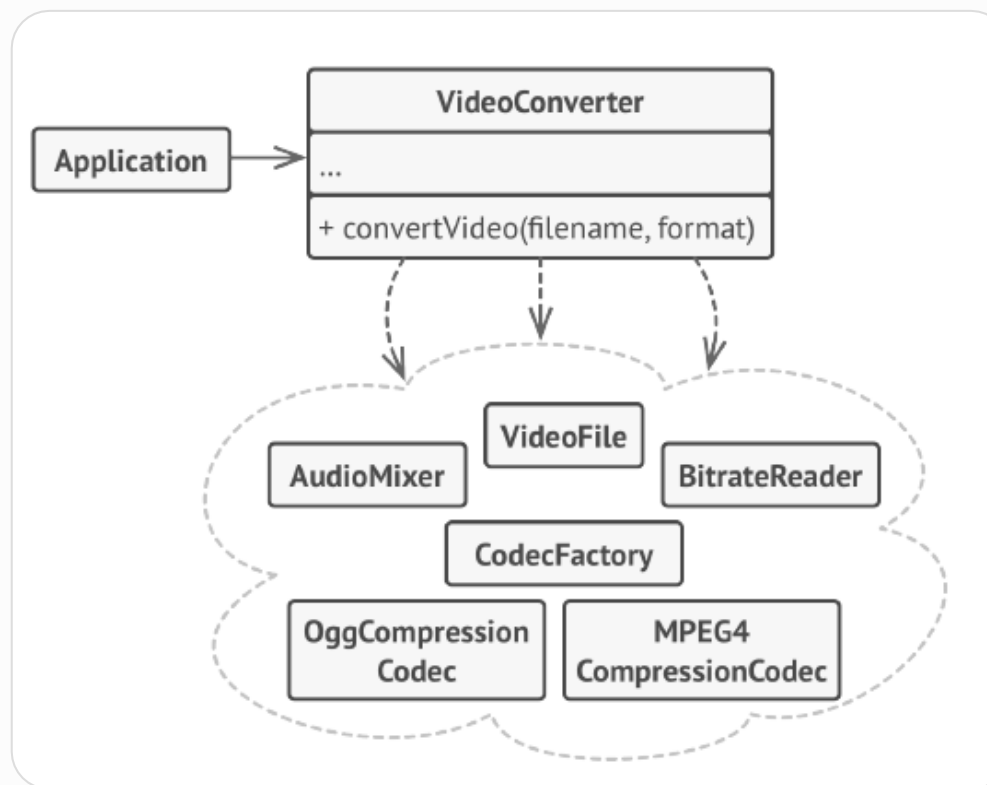
* Imagen capturada en el libro versión
html publicado en CD (Gamma et al 1995)



(Shvets, Alexander, 2018)

Patrón de Proyecto Fachada

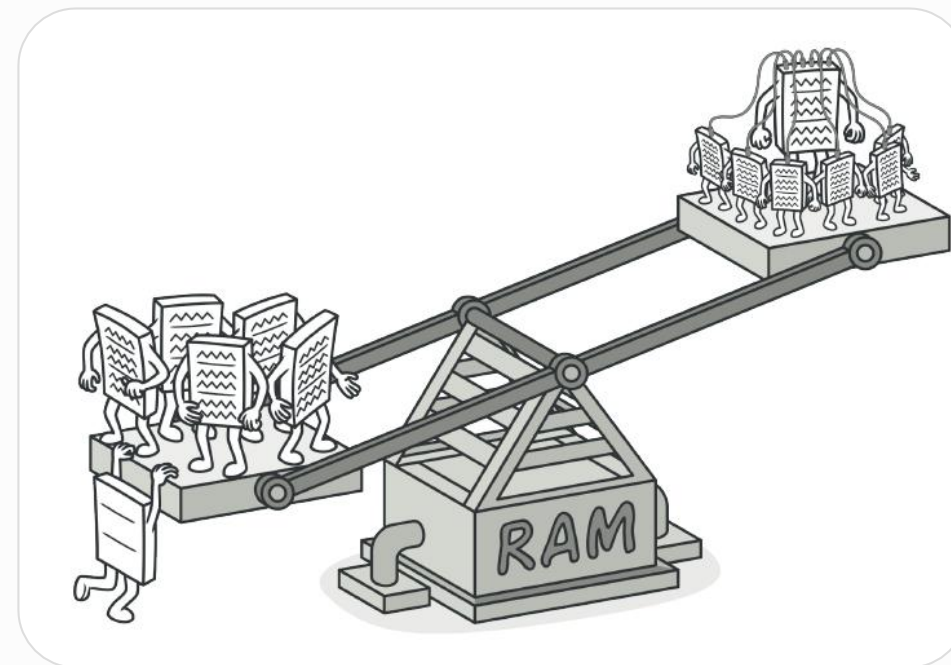
- Ejemplo:



(Shvets, Alexander, 2018)

Patrón de Proyecto Peso pluma (del inglés *Flyweight*)

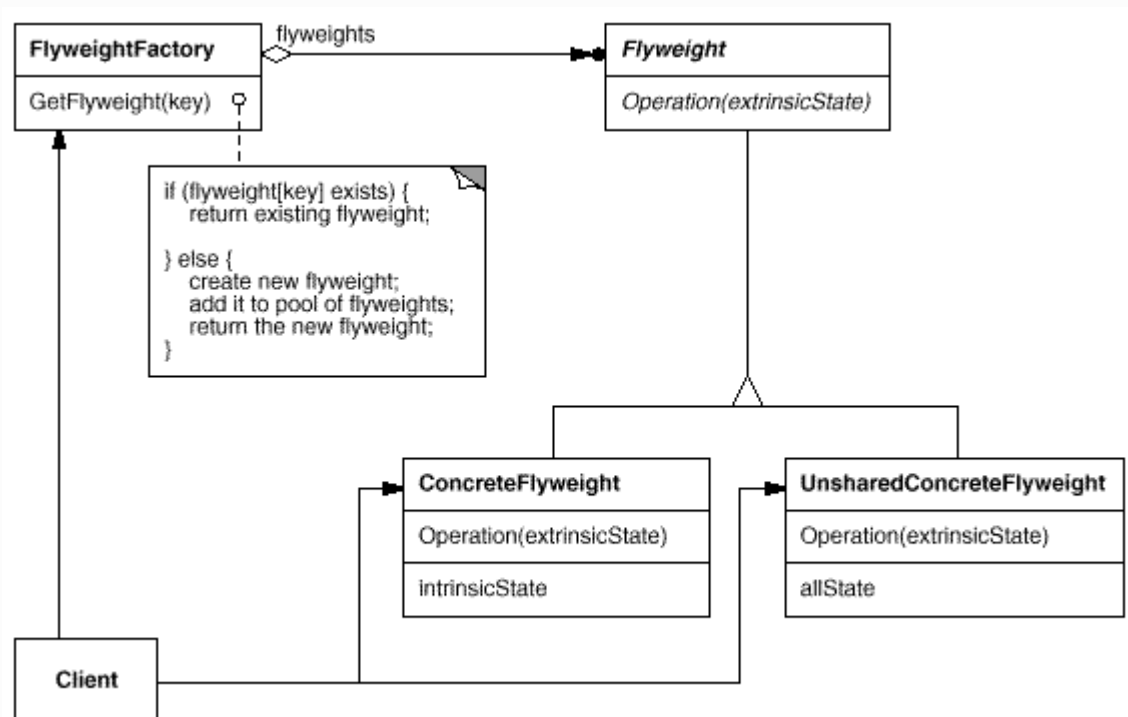
- **Clasificación:** Patrón estructural/Ámbito de objetos
- **Intención:** Usar el compartimiento para ofrecer soporte eficiente a un gran número de objetos de granularidad fina. Este patrón permite, por ejemplo, que usted encaje más objetos en la cantidad disponible de RAM compartiendo partes comunes de estado entre varios objetos en lugar de mantener todos los datos en cada objeto.
- **Aplicación:** Use el patrón Peso pluma solo si su programa necesita soportar un gran número de objetos similares que están sobrecargando la RAM disponible. Los objetos deben contener estados duplicados que pueden ser extraídos y compartidos entre varios objetos.



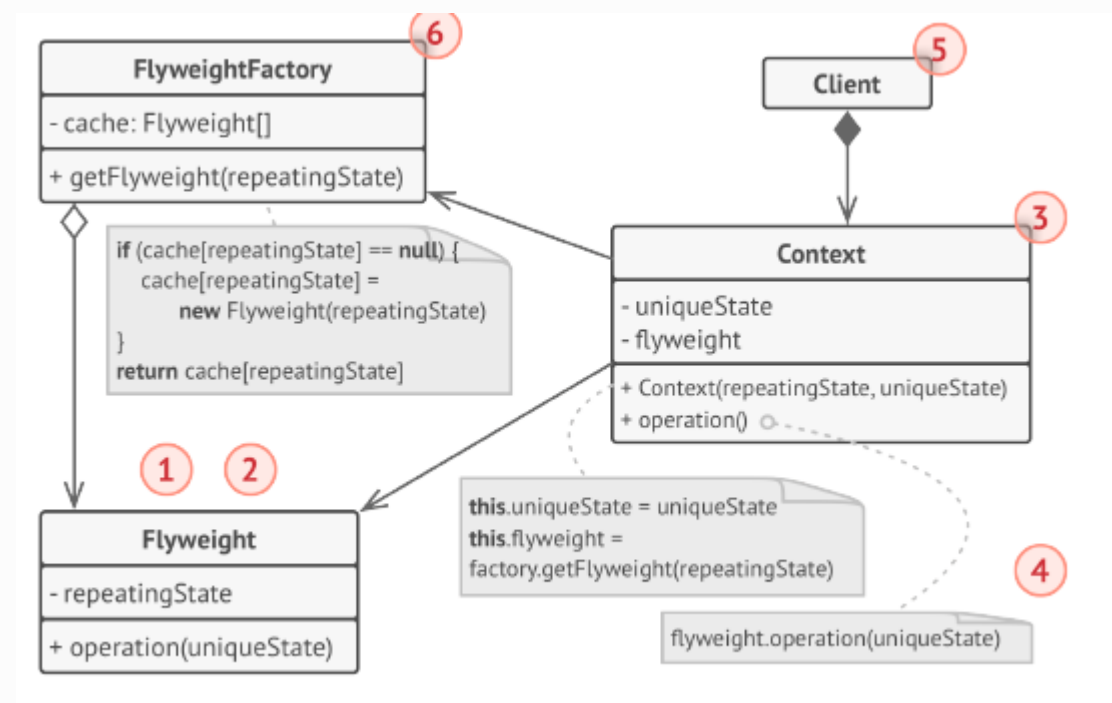
(Shvets, Alexander, 2018)

Patrón de Proyecto Peso pluma

- Estructura:



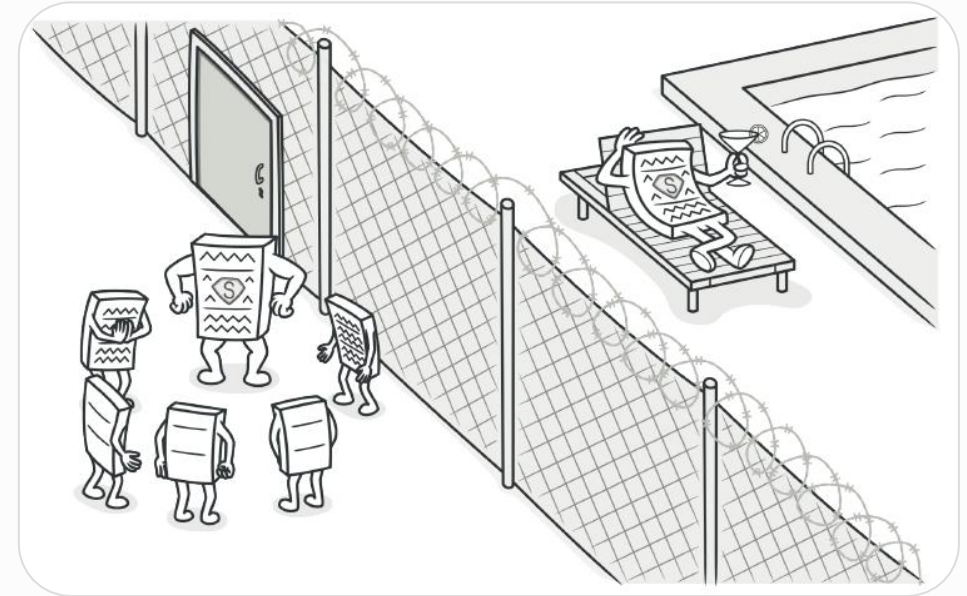
** Imagen capturada en el libro versión
html publicado en CD (Gamma et al 1995)



(Shvets, Alexander, 2018)

Patrón de Proyecto Proxy (del inglés *Proxy*)

- **Clasificación:** Patrón estructural/Ámbito de objetos
- **Intención:** Proporcionar un sustituto o marcador de posición para otro objeto para controlar el acceso a él.
- **Motivación:** Podemos querer controlar el acceso a un objeto, por ejemplo, para posponer el costo total de su creación e inicialización hasta que realmente necesitemos usarlo. El costo de creación del objeto en general se debe a que muchos atributos necesitan ser inicializados, además de las dependencias con otros objetos que componen el objeto.



(Shvets, Alexander, 2018)

Patrón de Proyecto *Proxy*

- **Aplicabilidad**
- Usos del patrón *Proxy*:
 - *Proxy* remoto: es necesario hacer referencia a un objeto en un espacio de direccionamiento diferente,
 - *Proxy* virtual: se desea crear objetos bajo demanda (posponer la materialización del objeto hasta que sea realmente necesaria), o “inicialización perezosa”
 - *Proxy* de protección: se desea mantener un mayor control sobre el objeto original (control de acceso, por ejemplo),
 - *Smart reference*: se desea mantener una referencia «inteligente» al objeto (el puntero al objeto realiza alguna acción adicional cada vez que se accesa al objeto).

Encuesta 7:
Utilización del *Proxy*



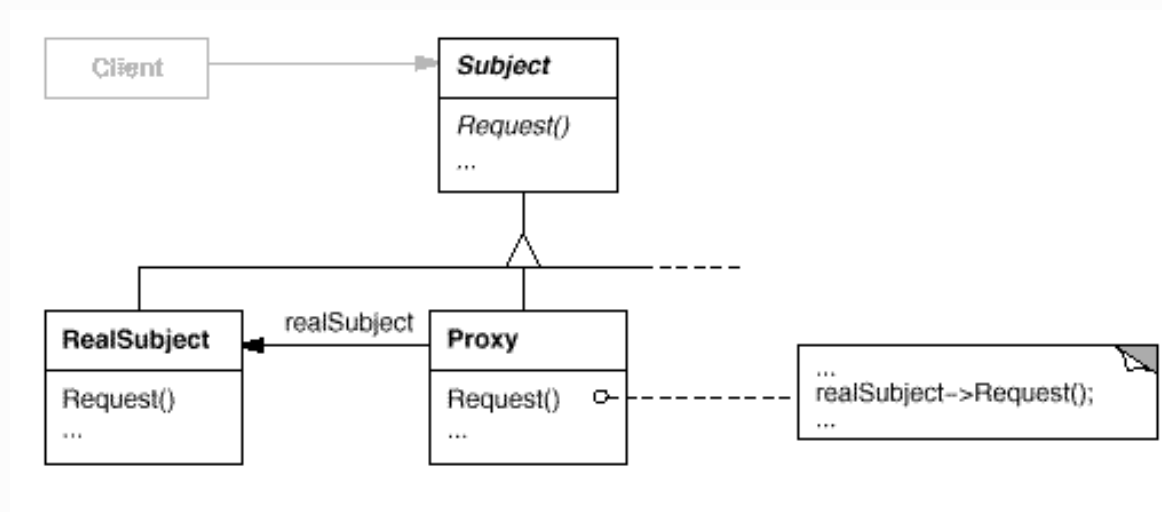
Patrón de Proyecto *Proxy*

- **Solución**

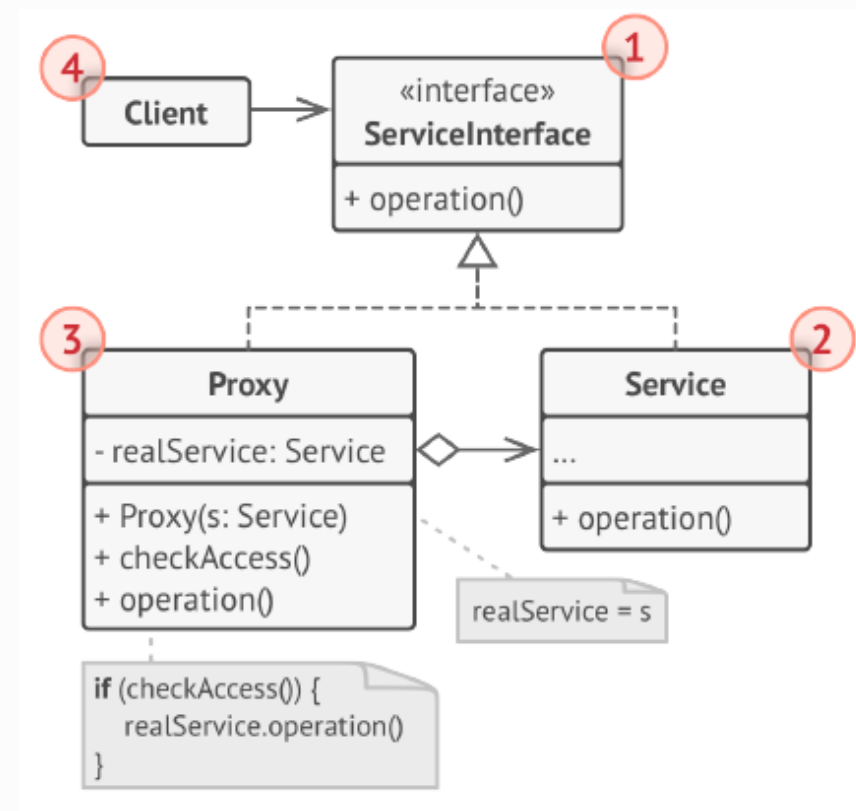
- Proporcione un sustituto, o apoderado, o incluso un marcador de posición para el objeto, con la finalidad específica de controlar el acceso al objeto.
- El Apoderado mantiene referencia al Sujeto Real y posee la misma interfaz para que pueda ser sustituido por el Sujeto Real.
- En general, es el *Proxy* quien crea el Sujeto Real, cuando es necesario.
- El *Proxy* delega solicitudes al Sujeto Real solamente cuando es necesario. Esto depende del tipo de *Proxy*, y varios tipos están descritos en la versión original del patrón Proxy (Gamma et al. 1995).

Patrón de Proyecto *Proxy*

- Estructura:



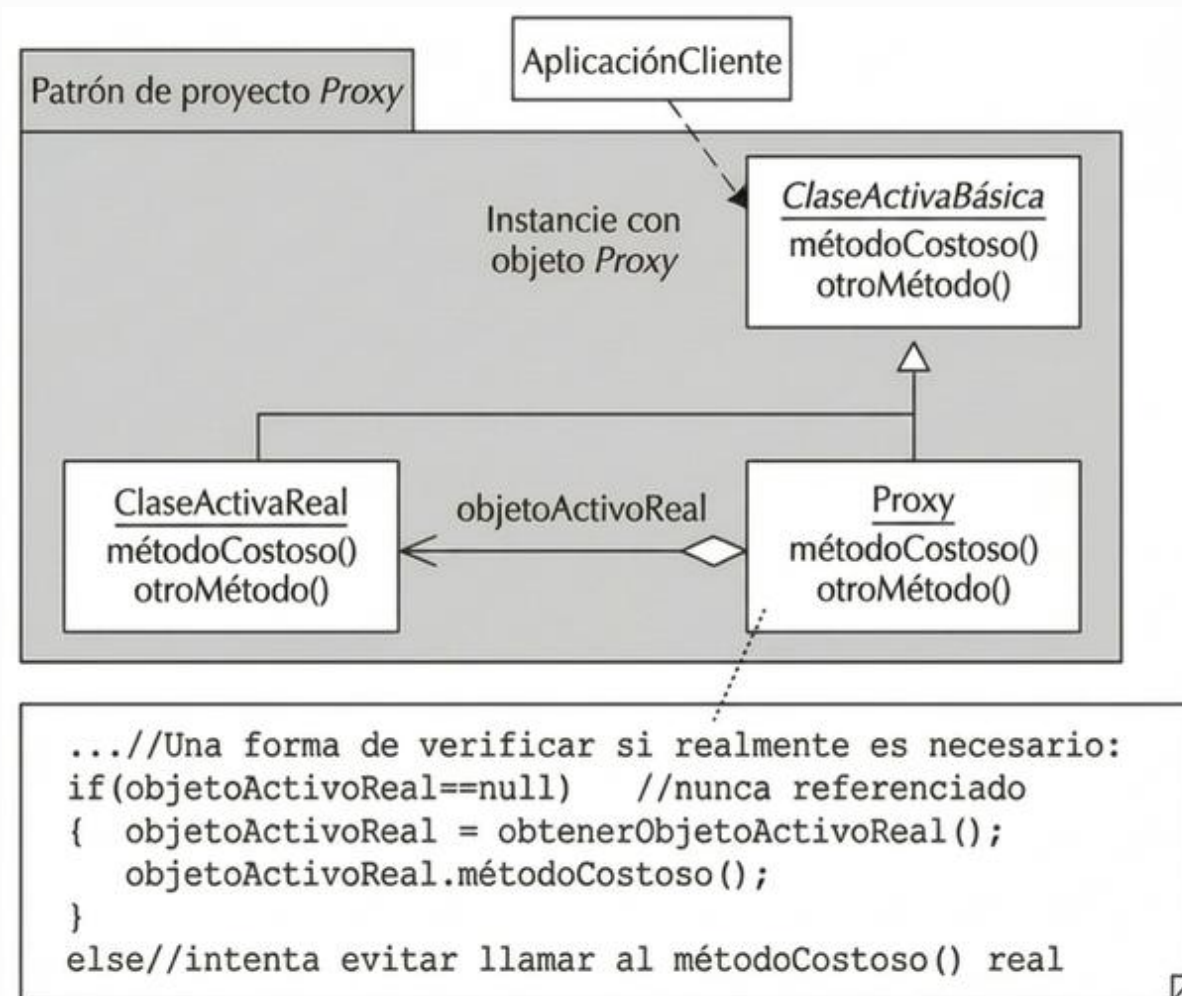
** Imagen capturada en el libro versión
html publicado en CD (Gamma et al 1995)



(Shvets, Alexander, 2018)

Patrón de Proyecto *Proxy*

- Estructura:

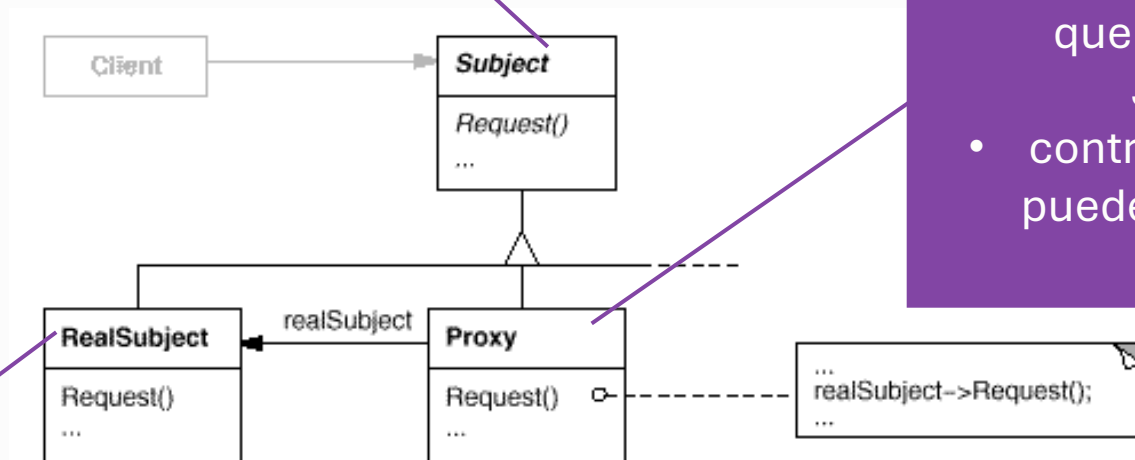


(Braude, Erick, 2005)

Patrón de Proyecto *Proxy*

- Participantes:

Define la interfaz común para *RealSubject* y *Proxy*, de modo que el *Proxy* pueda ser usado en lugar del *RealSubject*



- mantiene una referencia que permite que el *Proxy* accese a *RealSubject*, ya que las interfaces *RealSubject* y *Subject* son las mismas.
- controla el acceso a *RealSubject* y puede ser responsable de crearlo y eliminarlo.

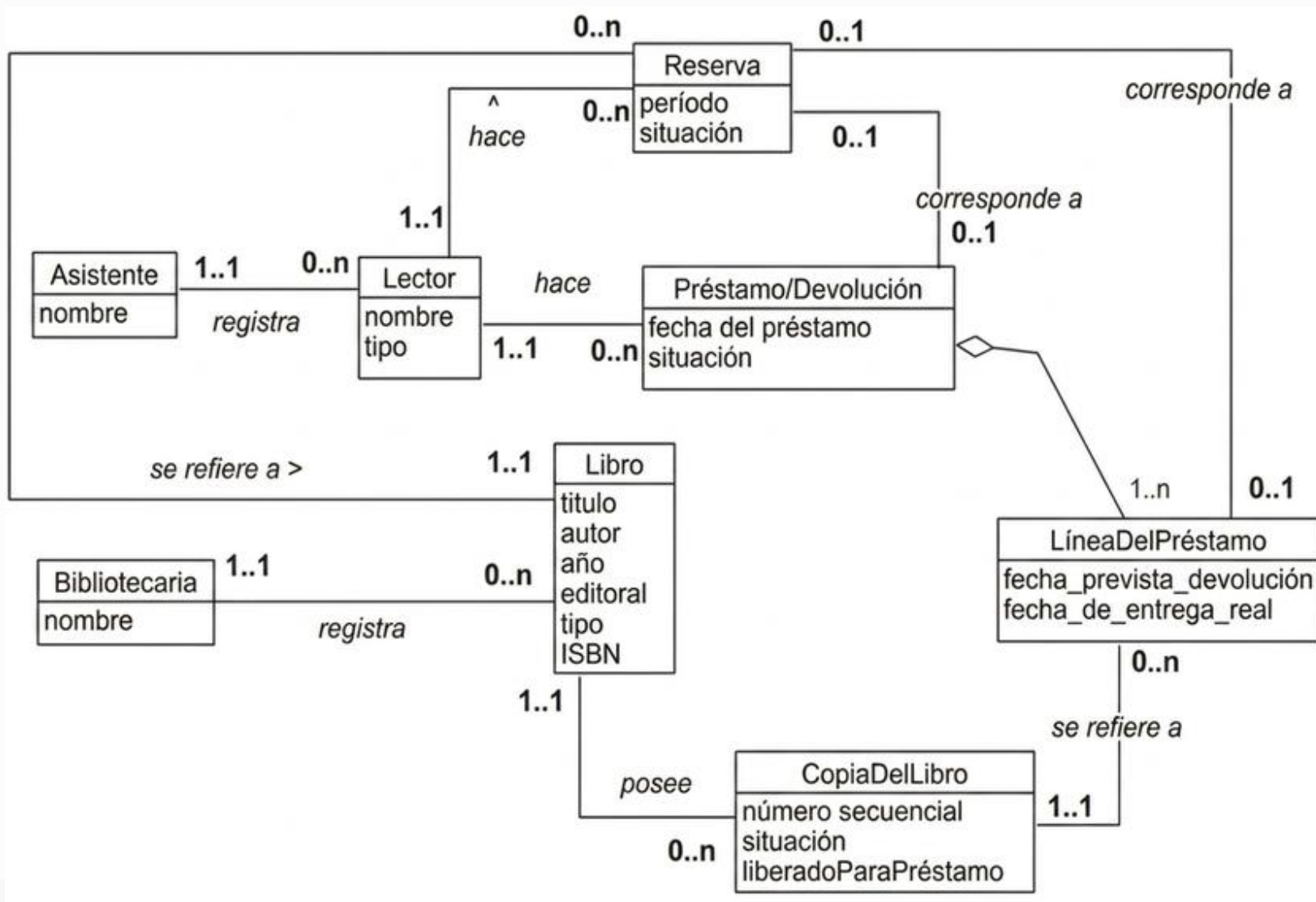
Define el *RealSubject* que el *proxy* representa

* Imagen capturada en el libro versión html publicado en CD (Gamma et al 1995)

Patrón de Proyecto *Proxy*

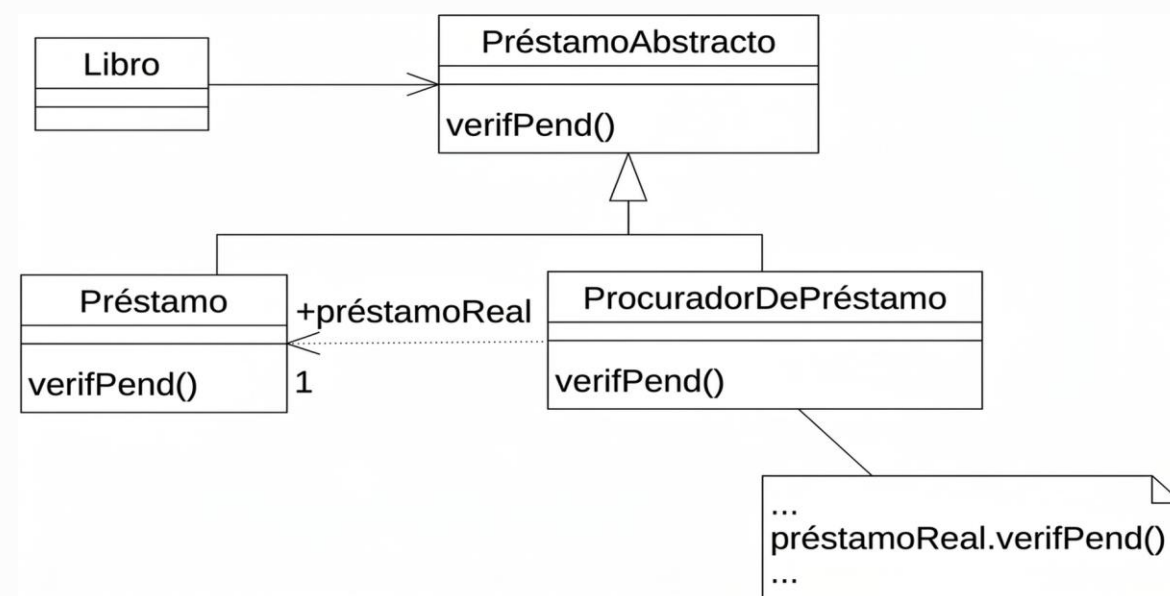
- **Ejemplo: materialización y desmaterialización de objetos**
 - Materialización: traer un registro de una tabla en forma de objeto
 - Desmaterialización: persistir un objeto como un registro en una tabla
- En un sistema de biblioteca, al materializar un lector, podemos al mismo tiempo materializar los objetos referentes a todos los préstamos de ese lector.
 - Sin embargo, los objetos préstamo quizás sean materializados inútilmente, ya que el sistema probablemente no utilizará información sobre esos objetos, desperdiciando tiempo y espacio.

Patrón de Proyecto *Proxy*



Patrón de Proyecto *Proxy*

- La variante *Proxy Virtual* puede usarse para solucionar este problema
- ¿Qué objetos asociados al objeto X deben recuperarse cada vez que se recupera un objeto X?
- Para los objetos que no necesitarán ser recuperados, se crea un *proxy*.
 - Ejemplo: al materializar un Libro, ¿debe materializarse también sus préstamos asociados?
 - Respuesta: no



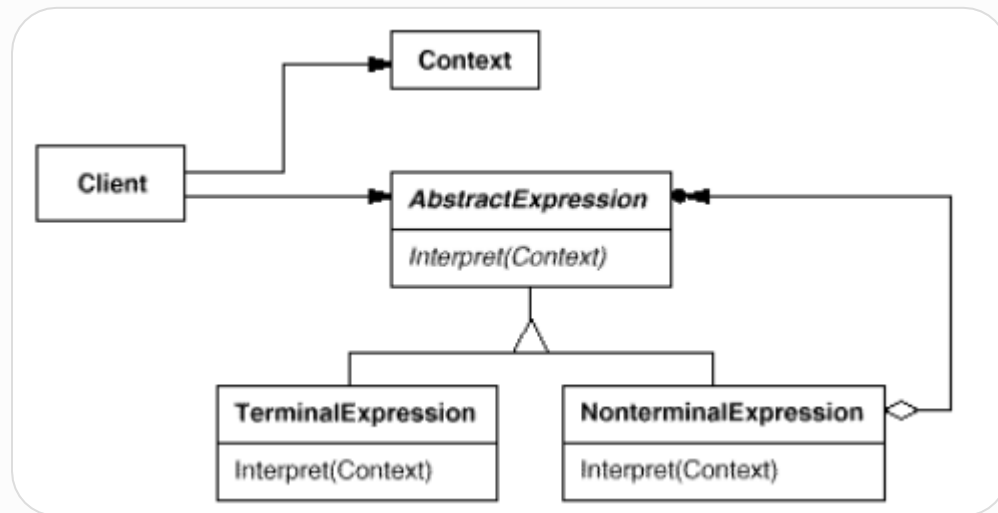
		Propósito		
		De creación	Estructurales	Conductual
Alcance	Clase	Método de fábrica	Adaptador	Intérprete Método plantilla
	Objeto	- Fábrica Abstracta - Constructor - Prototipo - Objeto Único (<i>Singleton</i>)	- Adaptador - Puente - Compuesto - Decorador - Fachada - Peso pluma <i>Proxy</i>	- Cadena de Responsabilidad - Comando - Iterador - Mediador - Memento - Observador - Estado - Estrategia - Visitante

PATRONES de la categoría “comportamental”

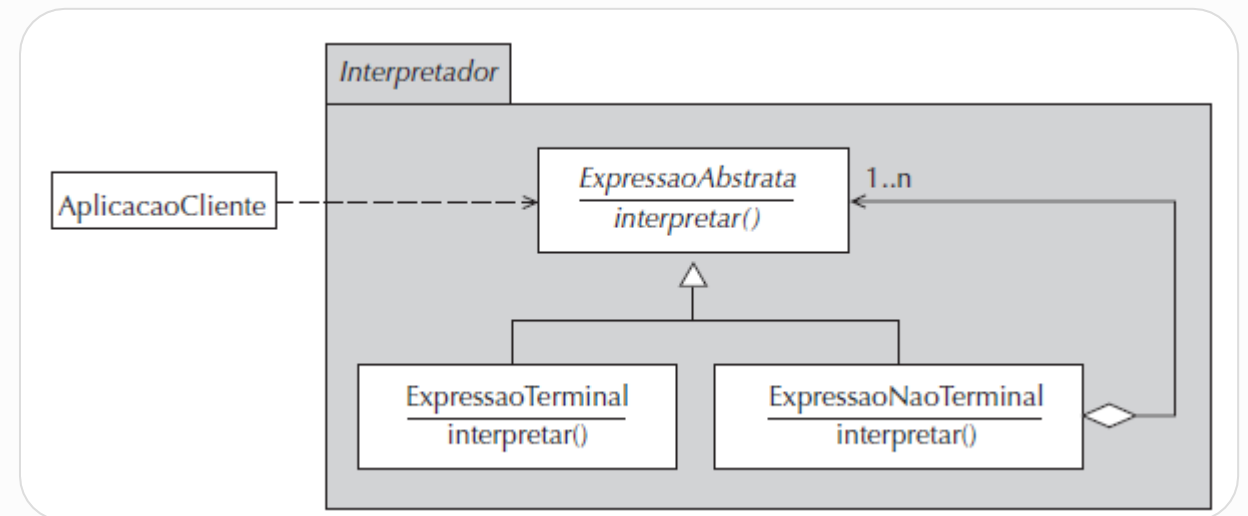
Responsables de caracterizar la forma en que las clases y objetos interactúan entre sí y distribuyen las responsabilidades

Patrón de Proyecto Intérprete (del inglés *Interpreter*)

- **Clasificación:** Patrón comportamental/Ámbito de clases
- **Intención:** Dado un lenguaje, definir una representación para su gramática junto con un intérprete que usa la representación para interpretar sentencias en el lenguaje.
- **Estructura:**



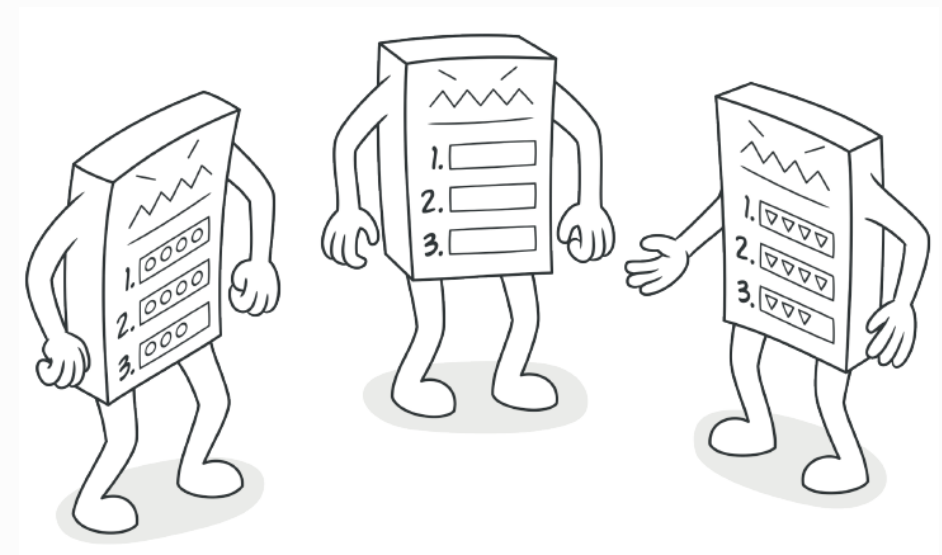
* Imagen capturada en el libro versión
html publicado en CD (Gamma et al 1995)



(Braude, Erick, 2005)

Patrón de Proyecto Método plantilla (del inglés *Template Method*)

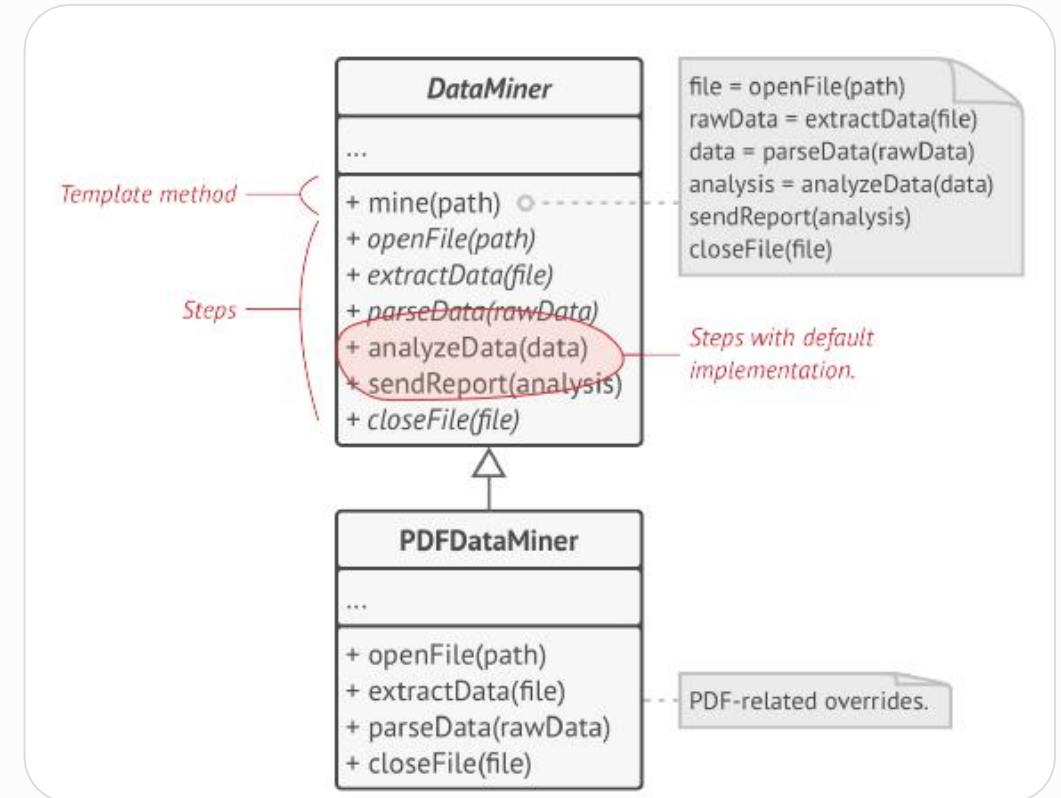
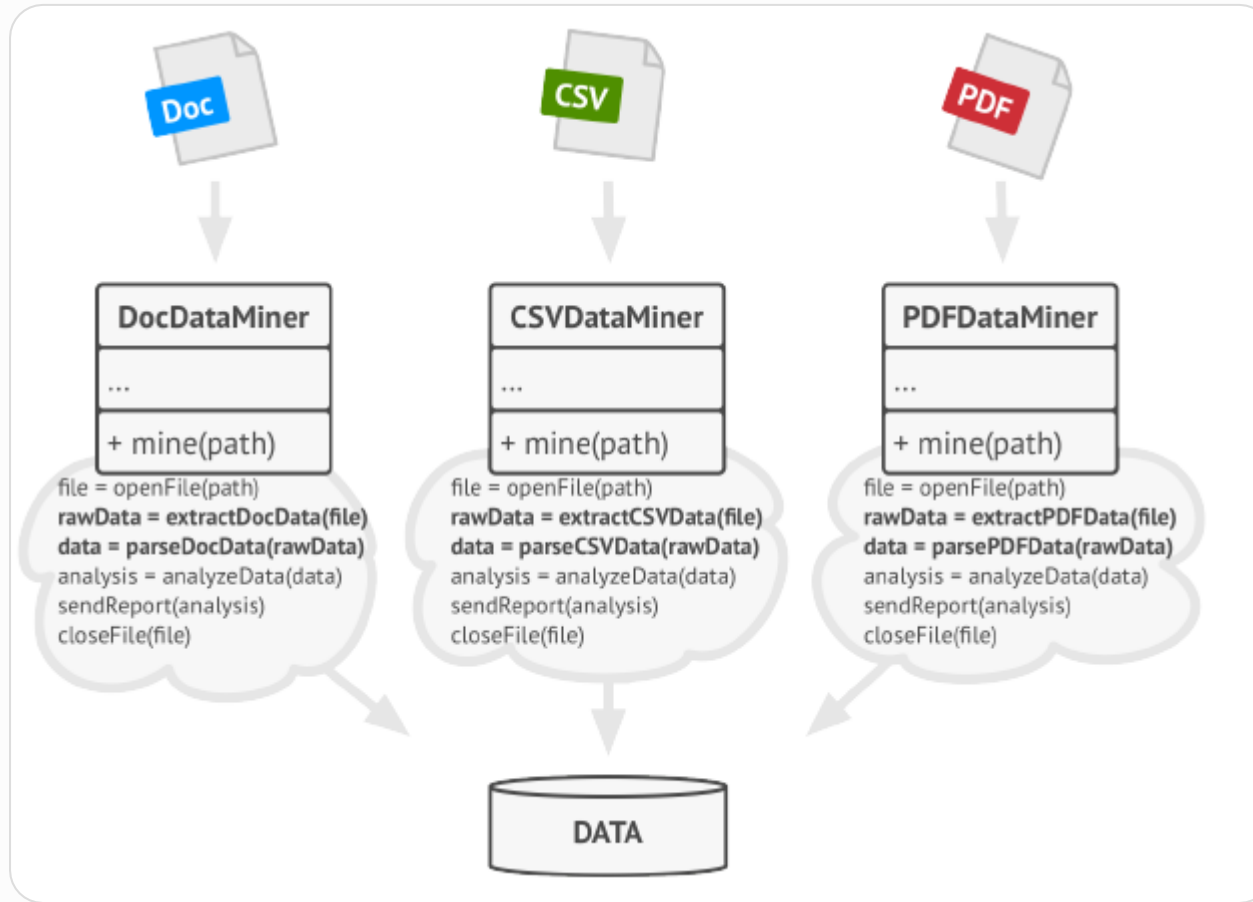
- **Clasificación:** Patrón comportamental/Ámbito de clases
- **Intención:** Definir el esqueleto de un algoritmo en una cierta operación, posponiendo algunos pasos para subclasses. El Método plantilla permite que las subclasses redefinan ciertos pasos de un algoritmo sin alterar la estructura del algoritmo.
 - La idea del Método plantilla es definir un método plantilla en una superclase, que contenga el esqueleto del algoritmo, con sus partes variables e invariables. Este método invoca otros métodos, algunos de los cuales son operaciones que pueden definirse en una subclase.
 - Así, las subclasses pueden redefinir los métodos que varían, añadiendo comportamiento específico de los puntos variables.



(Shvets, Alexander, 2018)

Patrón de Proyecto Método plantilla

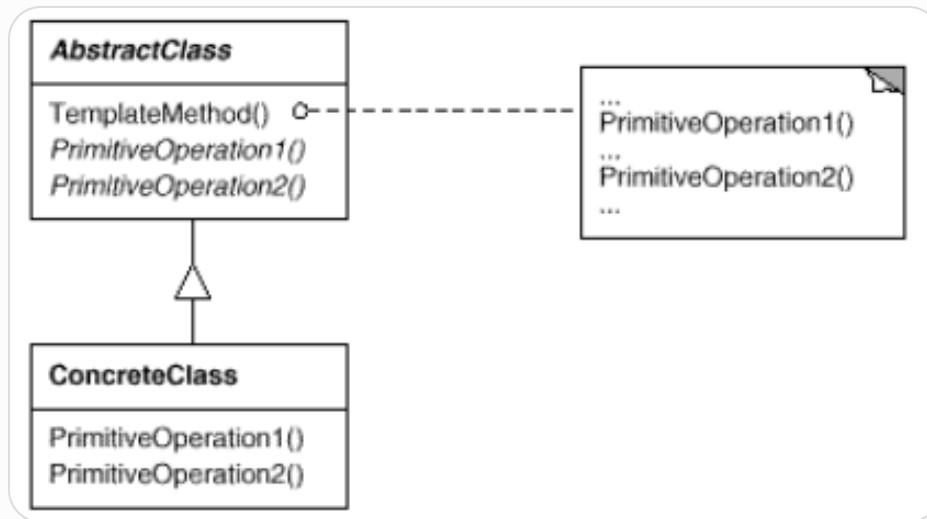
Aplicando el patrón ...



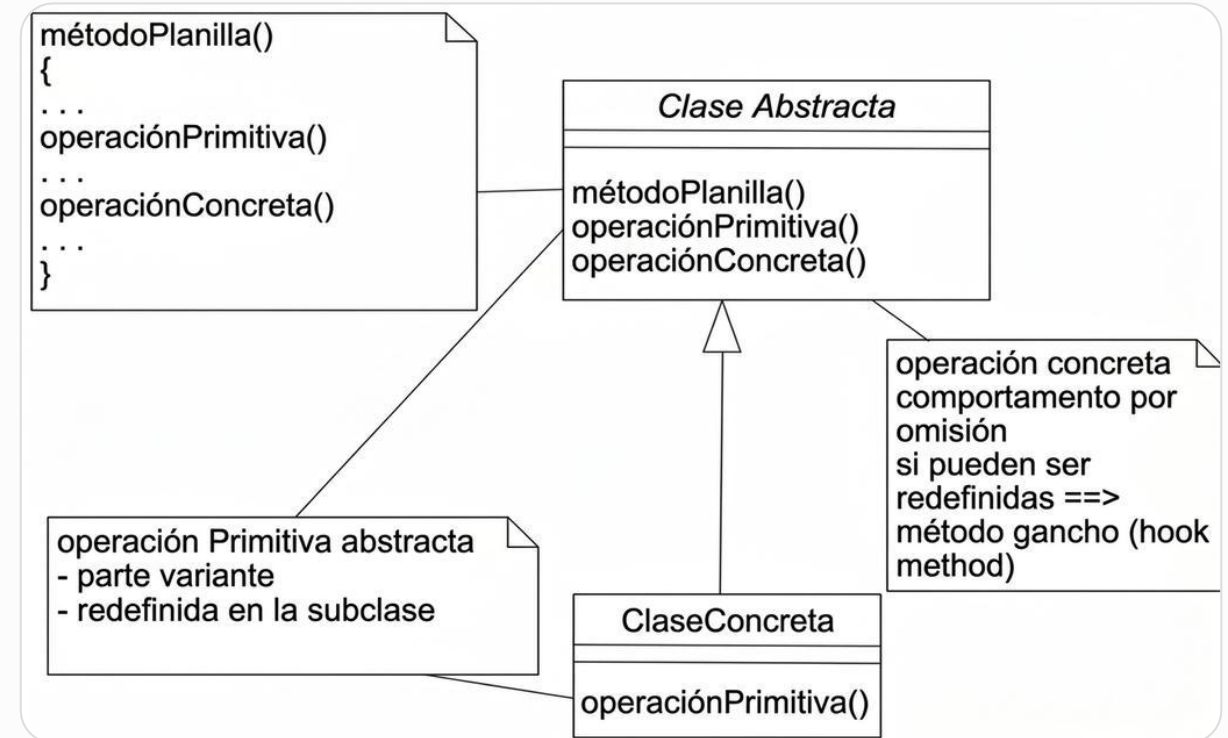
(Shvets, Alexander, 2018)

Patrón de Proyecto Método plantilla

- Estructura:



* Imagen capturada en el libro versión
html publicado en CD (Gamma et al 1995)



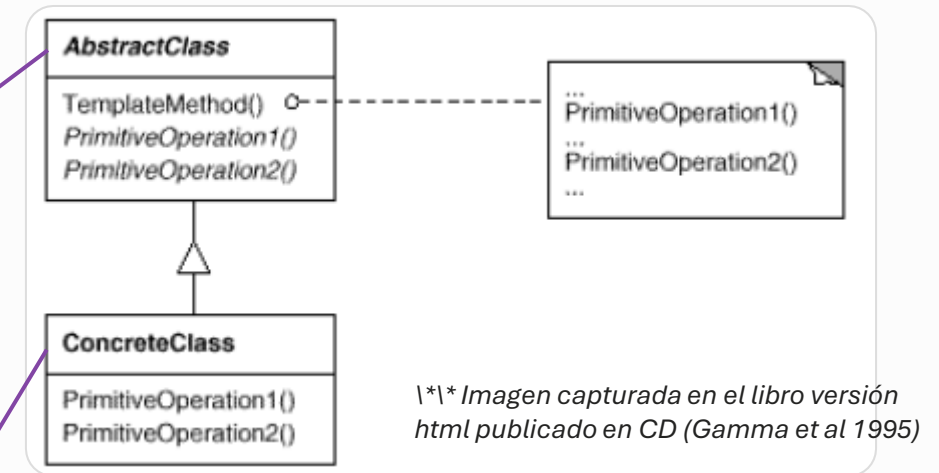
(Braude, 2005)

Patrón de Proyecto Método plantilla

- Participantes:

- define operaciones primitivas abstractas que las subclases concretas implementan como etapas de un algoritmo.
- implementa un método plantilla que define el esqueleto de un algoritmo. Llama a operaciones primitivas, así como a operaciones definidas en la clase abstracta o incluso en otros objetos.

- implementa las operaciones primitivas para ejecutar etapas específicas de la subclase del algoritmo.

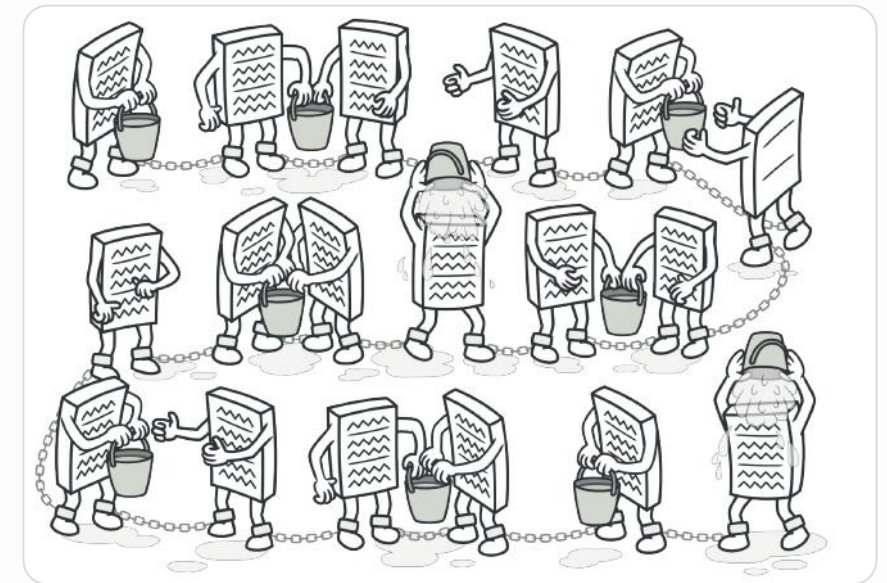


Encuesta 8:
¿Patrón
relacionado?



Patrón de Diseño Cadena de Responsabilidades (del inglés *Chain of Responsibilities*)

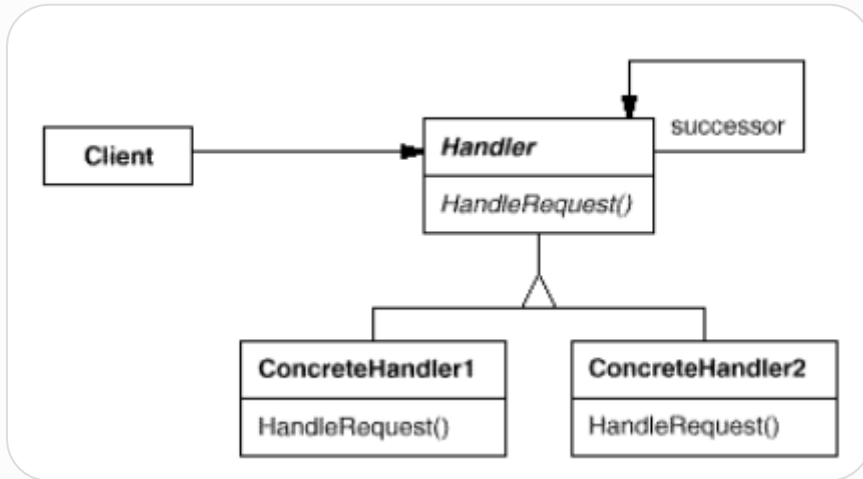
- **Clasificación:** Patrón de comportamiento/Ámbito de objetos
- **Intención:** Evitar acoplar el remitente de una solicitud a su destinatario dando a más de un objeto la oportunidad de manipular la solicitud. Encadenar los objetos receptores y pasar la solicitud a lo largo de la cadena hasta que un objeto la manipule.
- **Aplicabilidad:** use el patrón Cadena de Responsabilidades cuando su programa necesita procesar diferentes tipos de solicitudes de diversas formas, pero los tipos exactos de solicitudes y su secuencia no se conocen de antemano



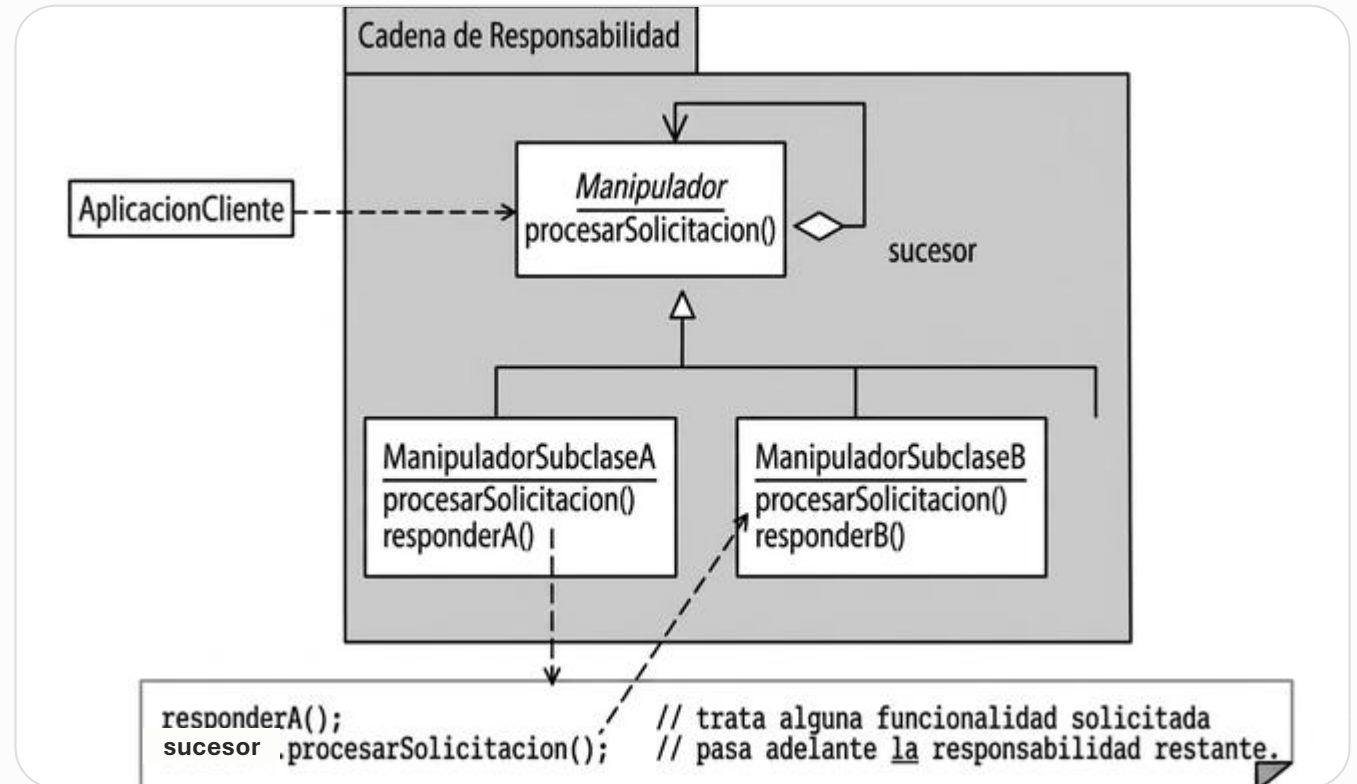
(Shvets, Alexander, 2018)

Patrón de Proyecto Cadena de Responsabilidades

- Estructura:



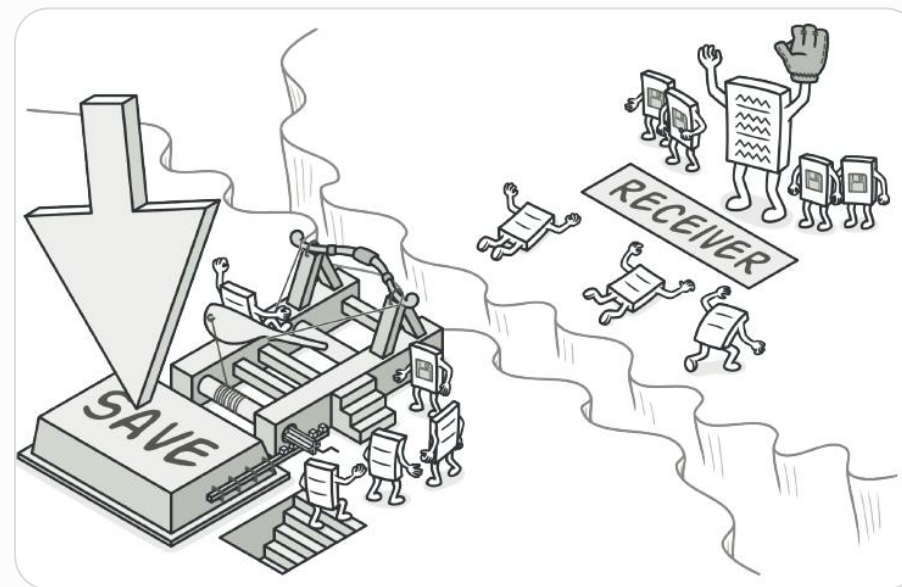
\|* Imagen capturada en el libro versión
html publicado en CD (Gamma et al 1995)



(Braude, 2005)

Patrón de Proyecto Comando (del inglés *Command*)

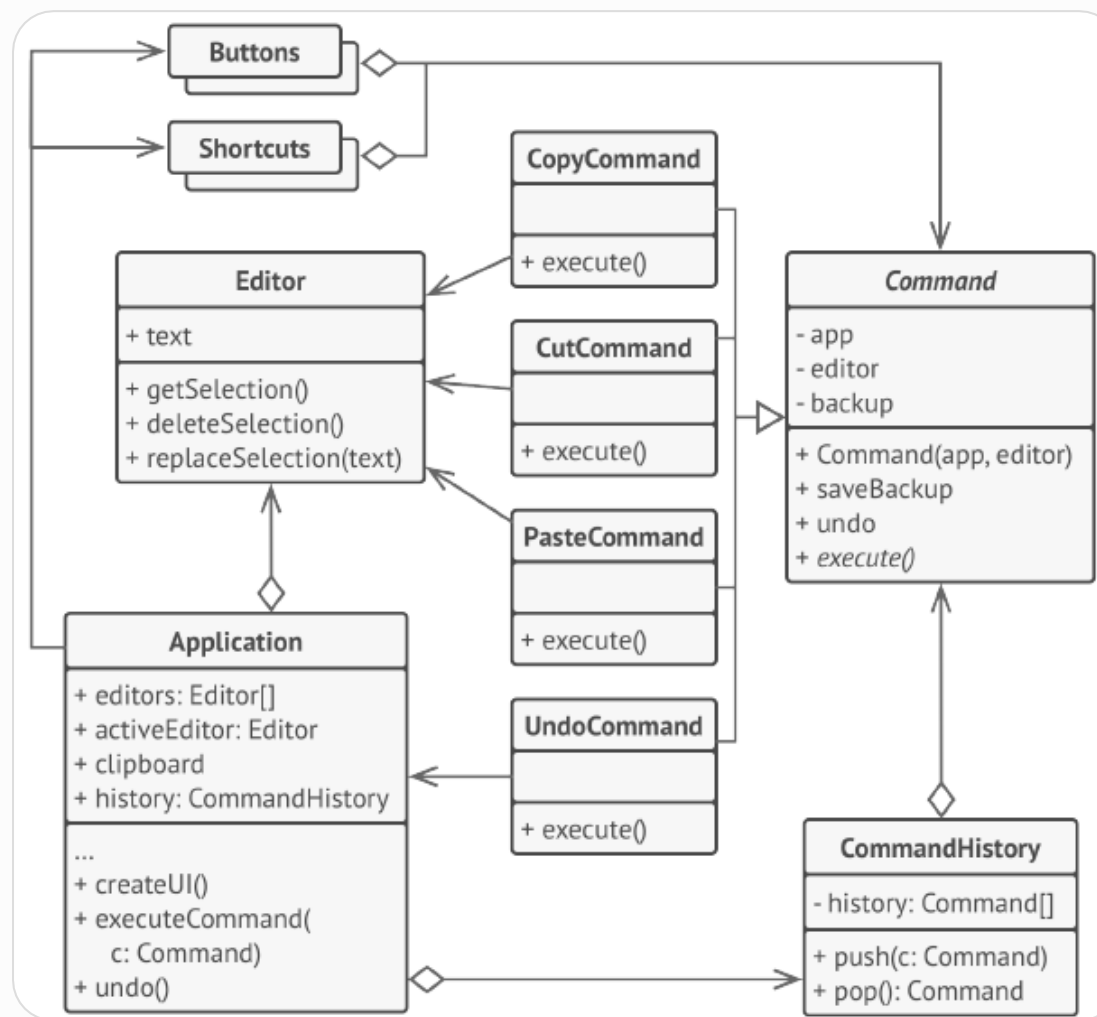
- **Clasificación:** Patrón de comportamiento/Ámbito de objetos
- **Intención:** Encapsular una solicitud como un objeto, permitiendo así parametrizar los clientes con diferentes solicitudes, crear colas o *logs* de solicitudes y dar soporte a operaciones que pueden deshacerse.
- **Motivación:** en algunas aplicaciones, puede que necesite parametrizar objetos con operaciones.



(Shvets, Alexander, 2018)

Patrón de Proyecto Comando

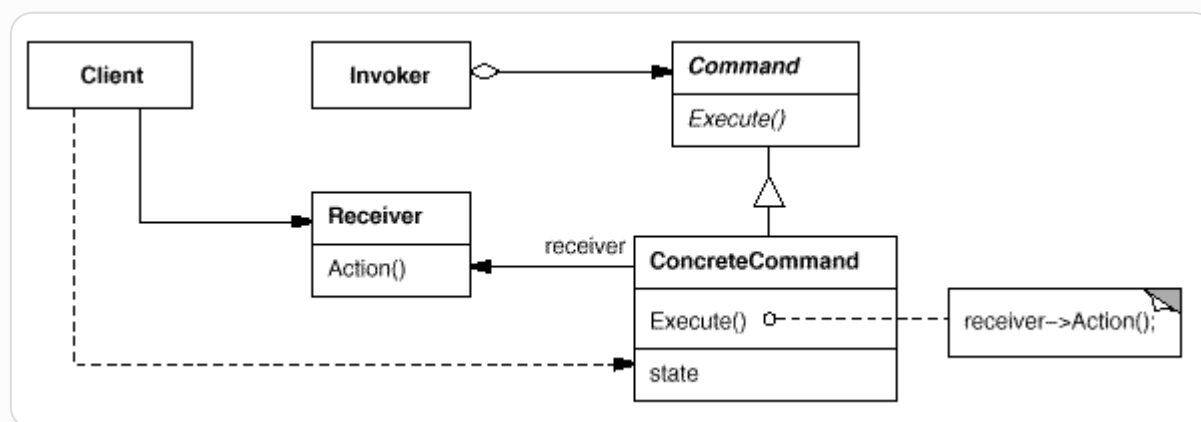
- Ejemplo:



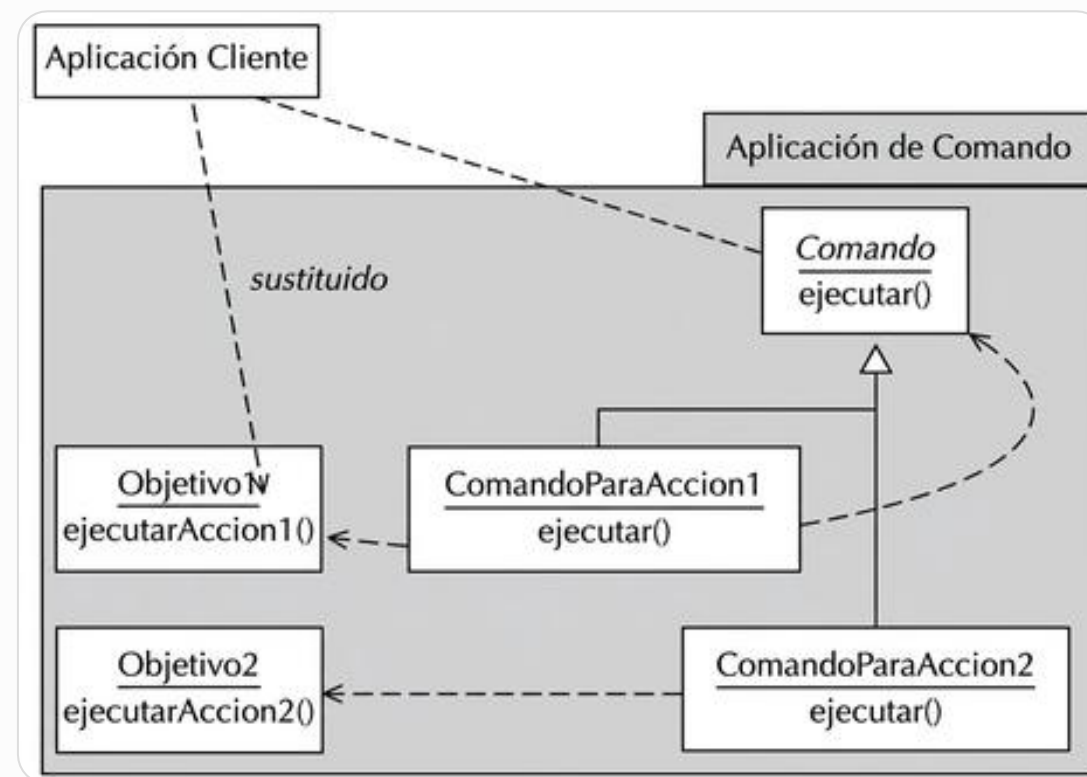
(Shvets, Alexander, 2018)

Patrón de Proyecto Comando

- Estructura:



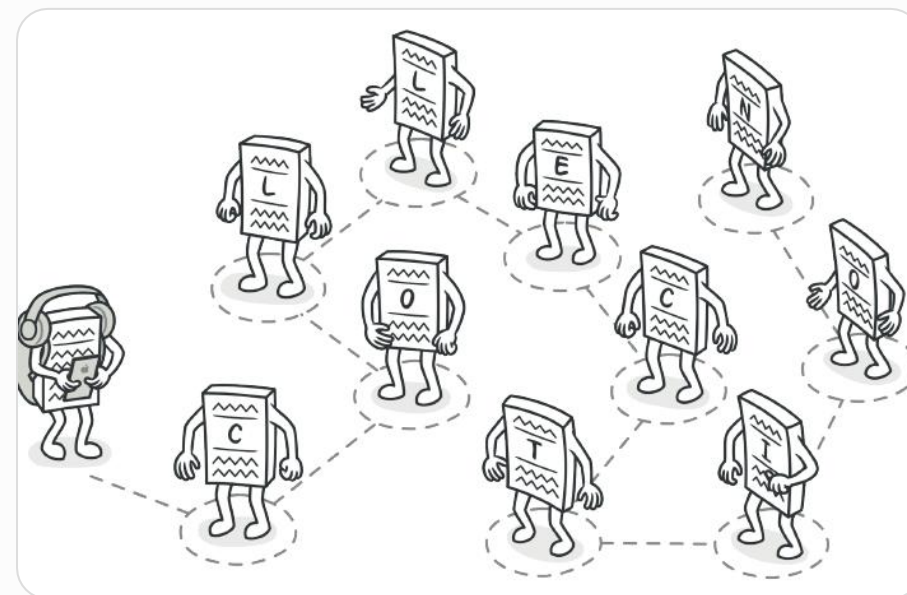
* Imagen capturada en el libro versión
html publicado en CD (Gamma et al 1995)



(Braude, 2005)

Patrón de Protección Iterador (del inglés *Iterator*)

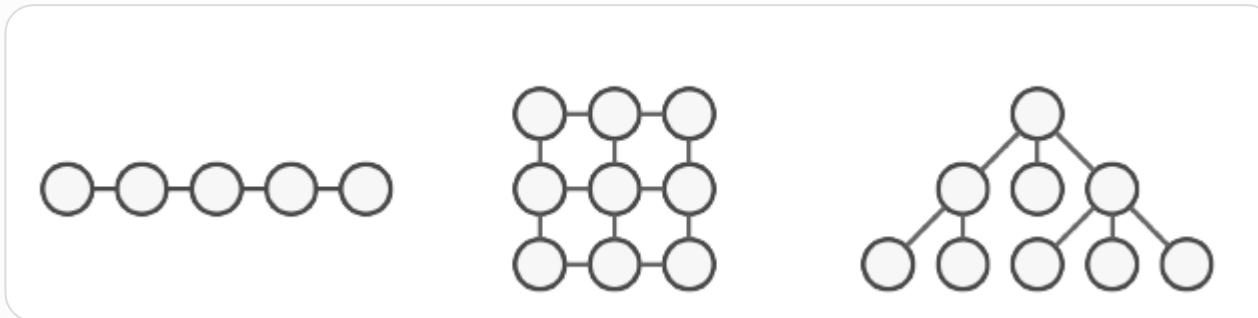
- **Clasificación:** Patrón de comportamiento/Ámbito de objetos
- **Intención:** Proporcionar una forma de acceder a los elementos de un objeto agregado secuencialmente sin exponer su representación subyacente.
- **Motivación:** Existe la necesidad de recorrer agregados cualesquiera, en diversos órdenes, pero cada uno tiene una forma diferente de ser recorrido. Incluir muchos métodos para encargarse de la iteración puede contaminar la interfaz. Si es deseable recorrer el agregado de diferentes formas, aún más métodos serán necesarios.



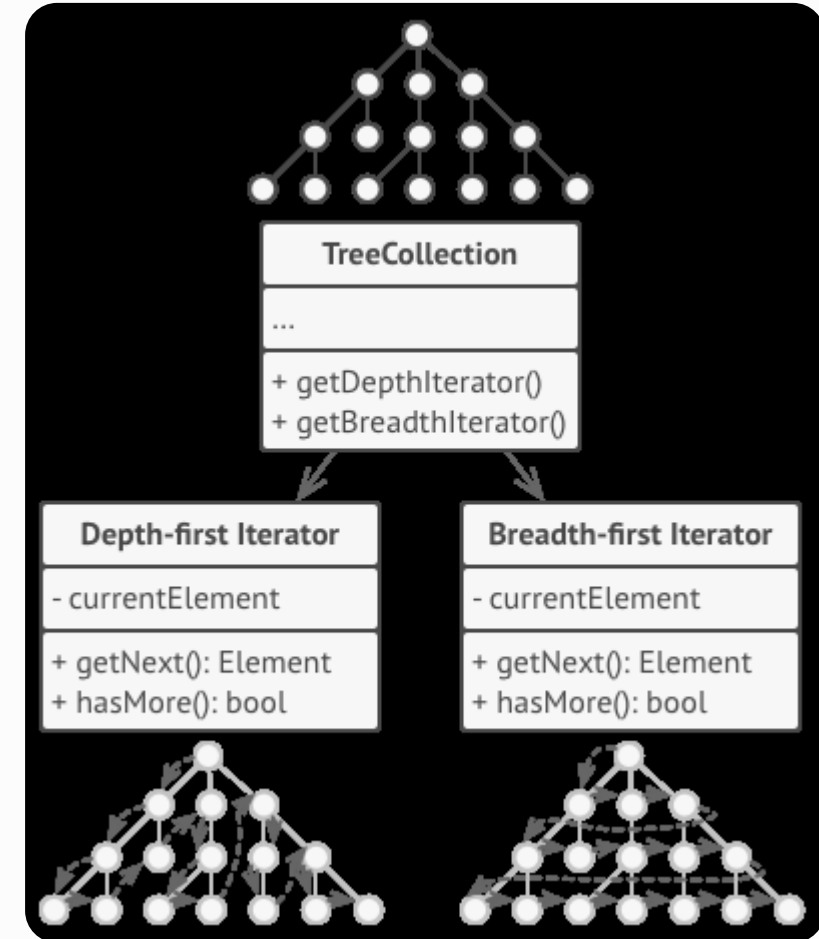
(Shvets, Alexander, 2018)

Patrón de Proyecto Iterador

- **Aplicabilidad:**
- Use el patrón Iterador para:
 - acceder al contenido de un objeto agregado sin exponer su representación interna.
 - dar soporte a múltiples recorridos de objetos agregados.
 - proporcionar una interfaz uniforme para recorrer diferentes estructuras agregadas (es decir, para apoyar la iteración polimórfica)

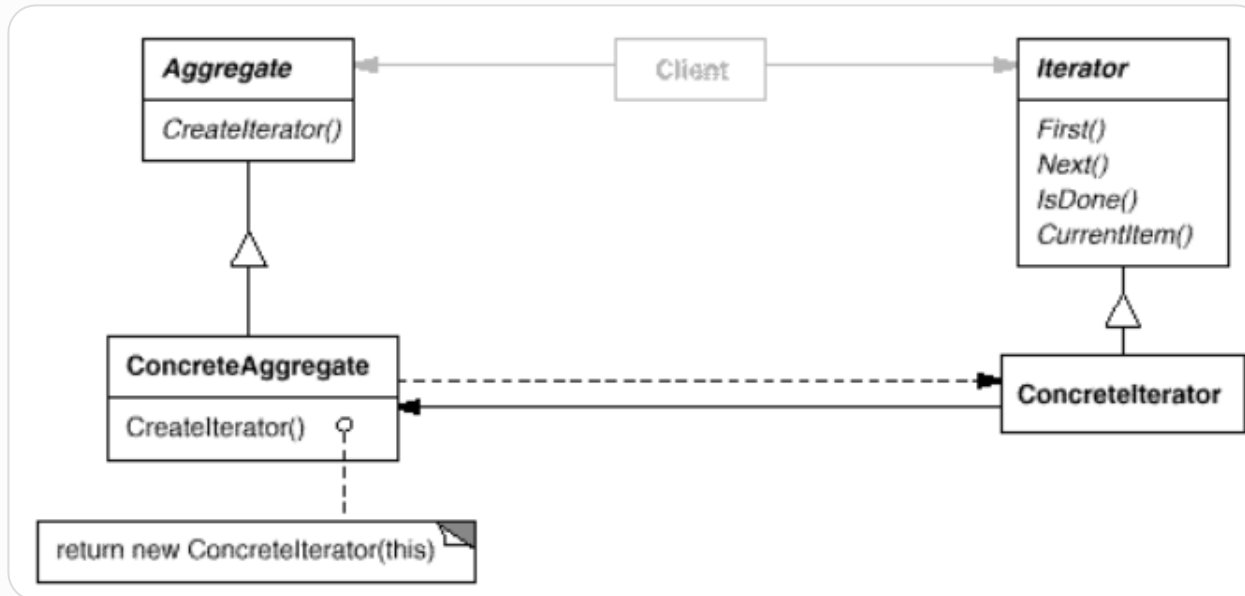


(Shvets, Alexander, 2018)

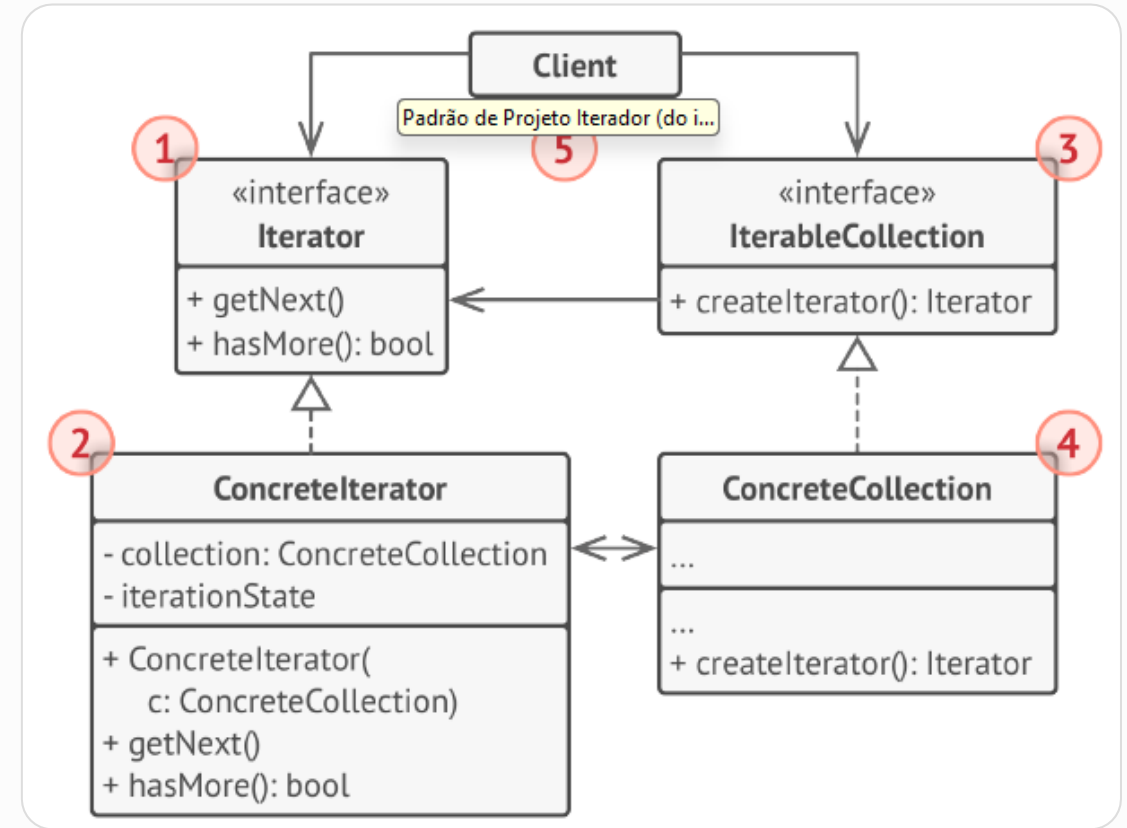


Patrón de Proyecto Iterador

- Estructura:



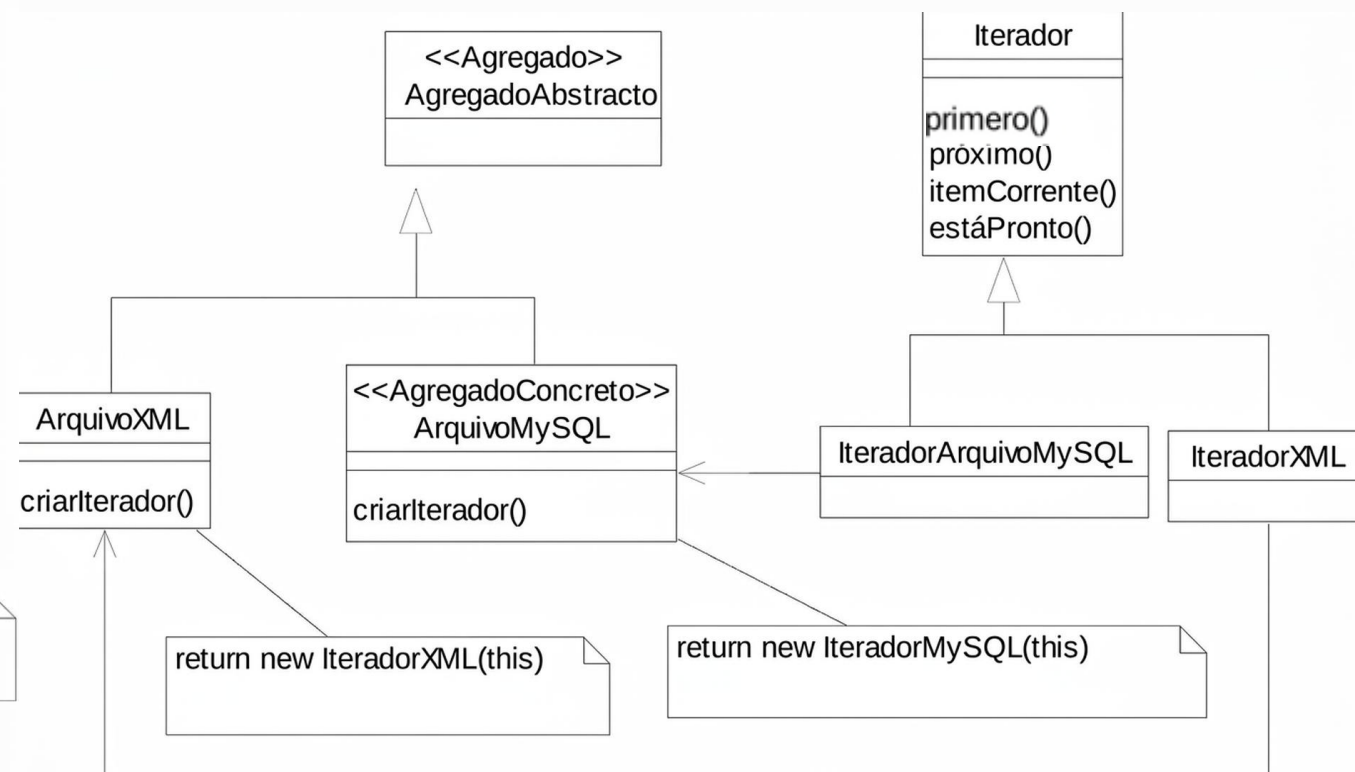
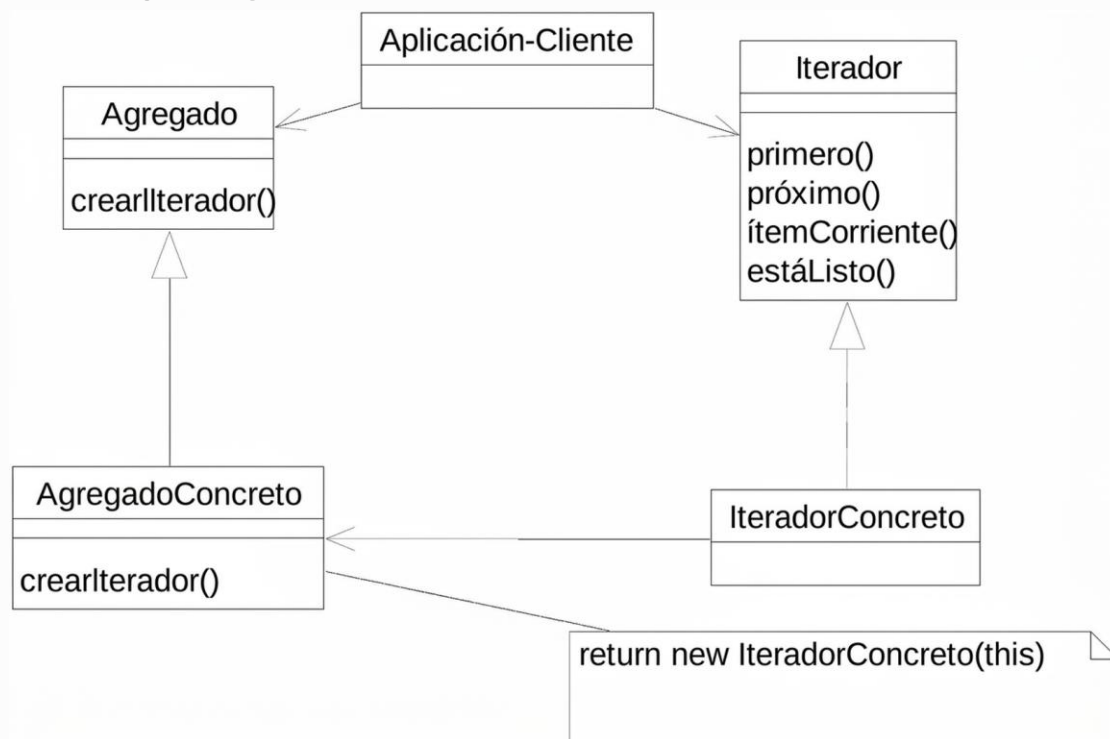
** Imagen capturada en el libro versión
html publicado en CD (Gamma et al 1995)



(Shvets, Alexander, 2018)

Patrón de Proyecto Iterador

- Ejemplo:

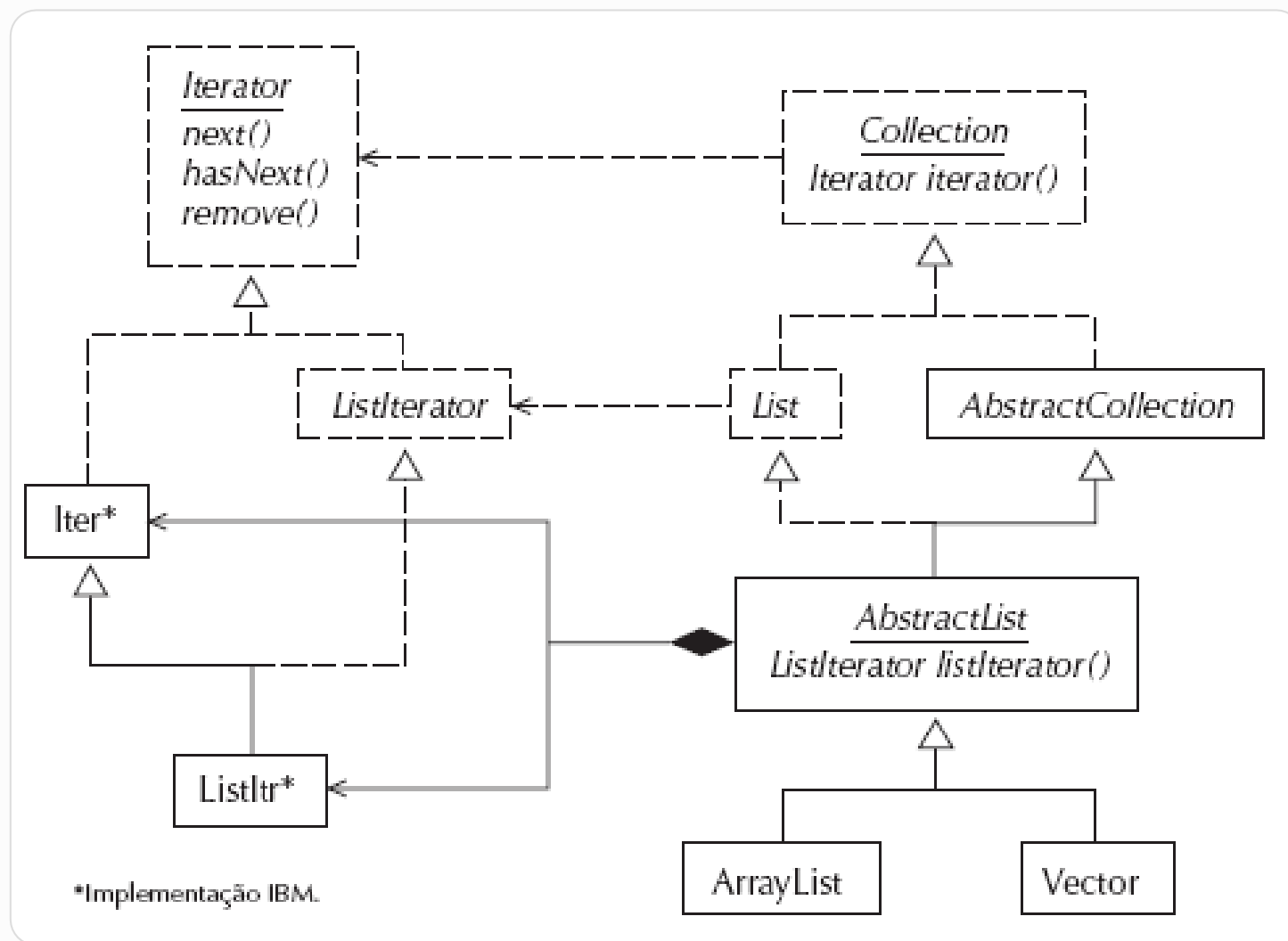


(Braude, 2005)

Patrón de Proyecto Iterador

- Iterador en la API Java
 - El paquete `java.util` contiene una interfaz `Iterator` con los métodos: `next()`, `hasNext()` y `remove()`.
 - El método `next()` es una combinación de `proximo()` e `itemCorrente()`
 - La subinterfaz `ListIterator` especifica estos métodos para operar en los objetos `List` y añade los métodos apropiados a los objetos `List`.
 - La interfaz `List` tiene un método `listIterator()`, que devuelve un objeto `ListIterator` que itera sobre él. Esto permite al programador moverse e iterar sobre objetos `List`.

Patrón de Proyecto Iterador

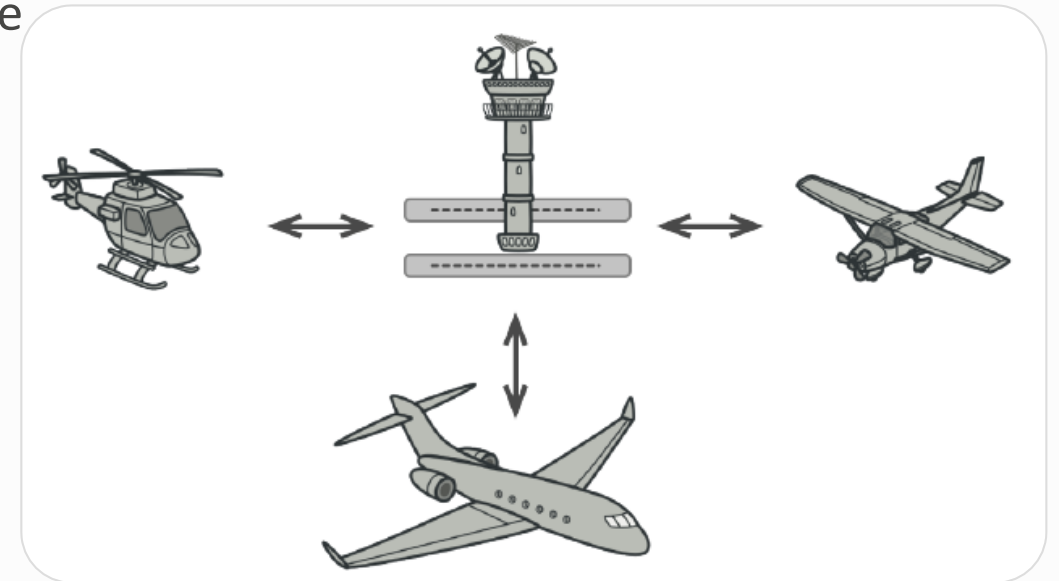


Patrón de Proyecto Iterador

```
public class Leitor
{
private String nome;
private Char[] tipo;
private Boolean lo ha encontrado=false;
private List préstamos\= new ArrayList();
public void devolverCopia(int codCopia)
{
    Iterator i = préstamos.iterator();
    while (i.hasNext()) && (!ha encontrado) {
        Emprestimo e = (Préstamo) i.next();
        achou=e.devolverCopia(codCopia) }
    }
}
```

Patrón de Proyecto Mediador (del inglés *Mediator*)

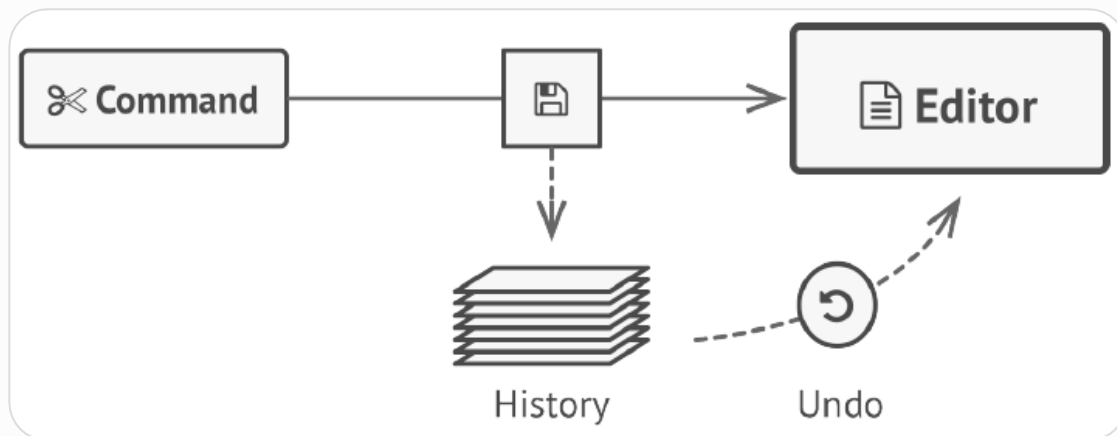
- **Clasificación:** Patrón de comportamiento/Ámbito de objetos
- **Intención:** Definir un objeto que encapsula la forma en que un conjunto de objetos interactúa. El Mediador promueve un acoplamiento débil al impedir que los objetos se refieran explícitamente entre sí, y permite variar su interacción de forma independiente.



(Shvets, Alexander, 2018)

Patrón de Proyecto Memento

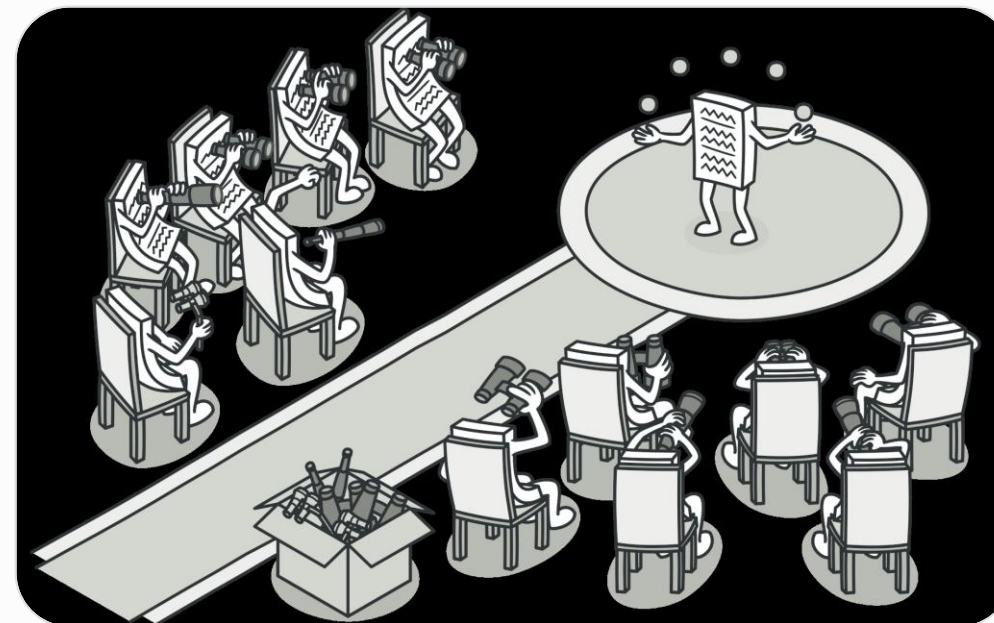
- **Clasificación:** Patrón de comportamiento/Ámbito de objetos
- **Intención:** Sin violar el encapsulamiento, capturar y externalizar el estado interno de un objeto para que el objeto pueda ser restaurado a ese estado posteriormente.



(Shvets, Alexander, 2018)

Patrón de Proyecto Observador (en inglés *Observer*)

- **Clasificación:** Patrón de comportamiento/Ámbito de objetos
- **Intención:** Definir una dependencia de uno a muchos entre objetos para que, cuando un objeto cambie de estado, todos sus dependientes sean notificados y actualizados automáticamente.

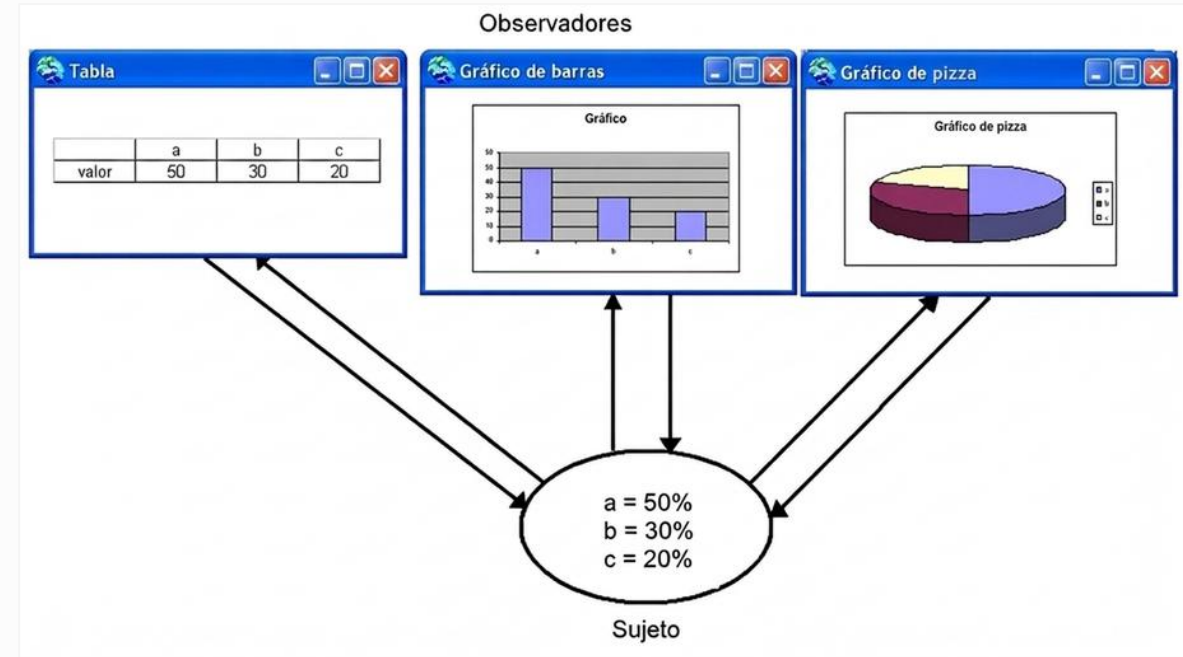


(Shvets, Alexander, 2018)

Patrón de Proyecto Observador

- **Aplicabilidad:**

- sistemas cuyas informaciones están preferentemente centralizadas en una única fuente (sujeto), y existen diversas vistas (observadores) que deben ser actualizadas siempre que la información sea modificada
 - Cualquier intento de mantener la consistencia aumenta el acoplamiento entre las clases.
 - Por ejemplo, en una herramienta integrada, se pueden separar los datos en sí de las diversas vistas sobre esos datos, de forma que un cambio en una de las vistas se refleje en las demás, incluso si las vistas no tienen conocimiento unas de otras.
- Cuando un objeto debe ser capaz de notificar a otros objetos sin hacer suposiciones sobre quiénes son esos objetos, o sin conocerlos de antemano o cuando esos objetos observadores cambian dinámicamente



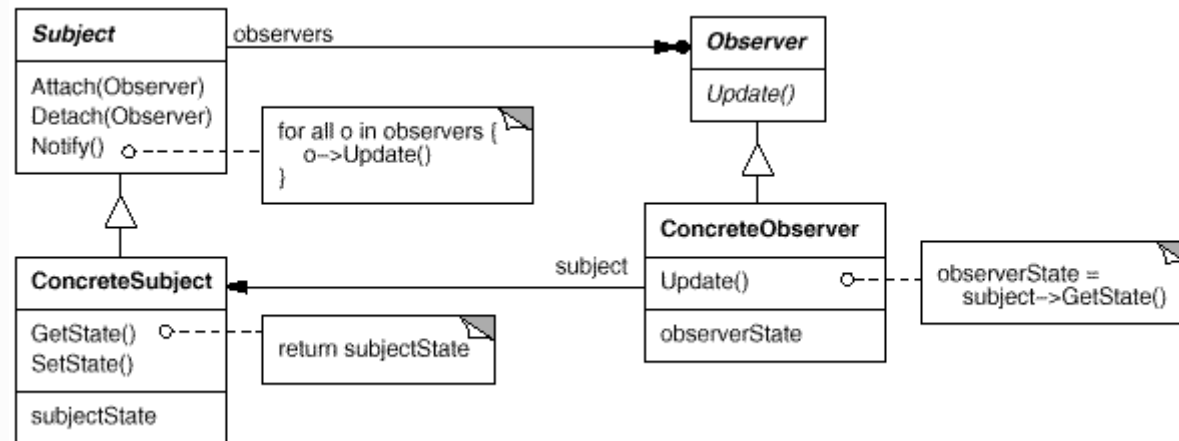
Encuesta 9:

¿A qué le recuerda esto?

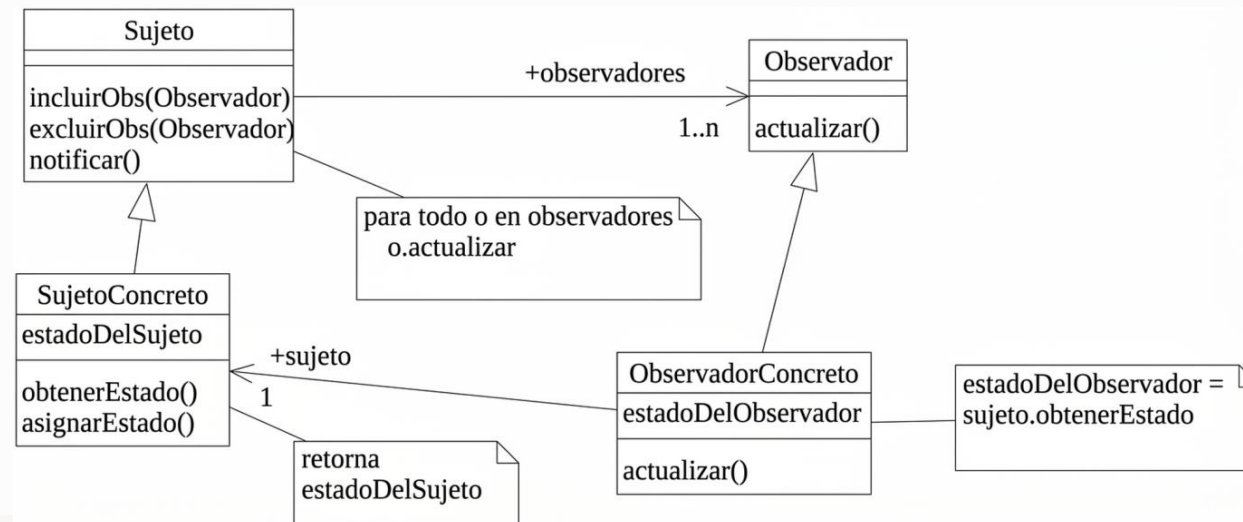


Patrón de Proyecto Observador

- Estructura:



\|* Imagen capturada en el libro versión
html publicado en CD (Gamma et al 1995)

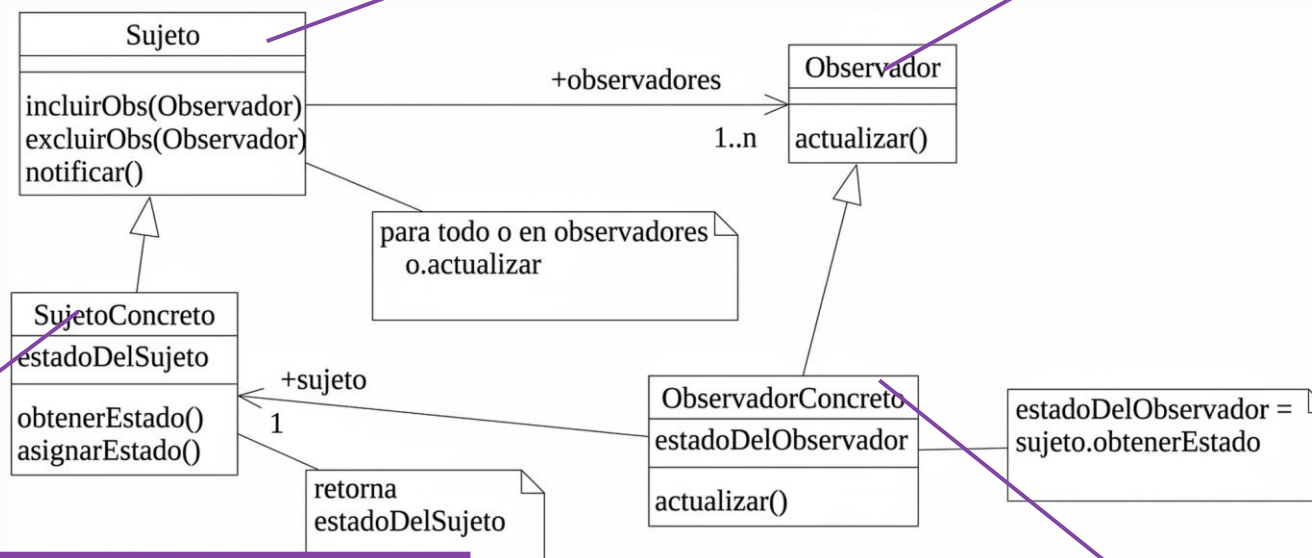


Patrón de Proyecto Observador

- **Participantes:**

Conoce a sus observadores, proporciona una interfaz para incluirlos y excluirllos

Define una interfaz de actualización para objetos que necesitarán ser notificados de cambios en el sujeto

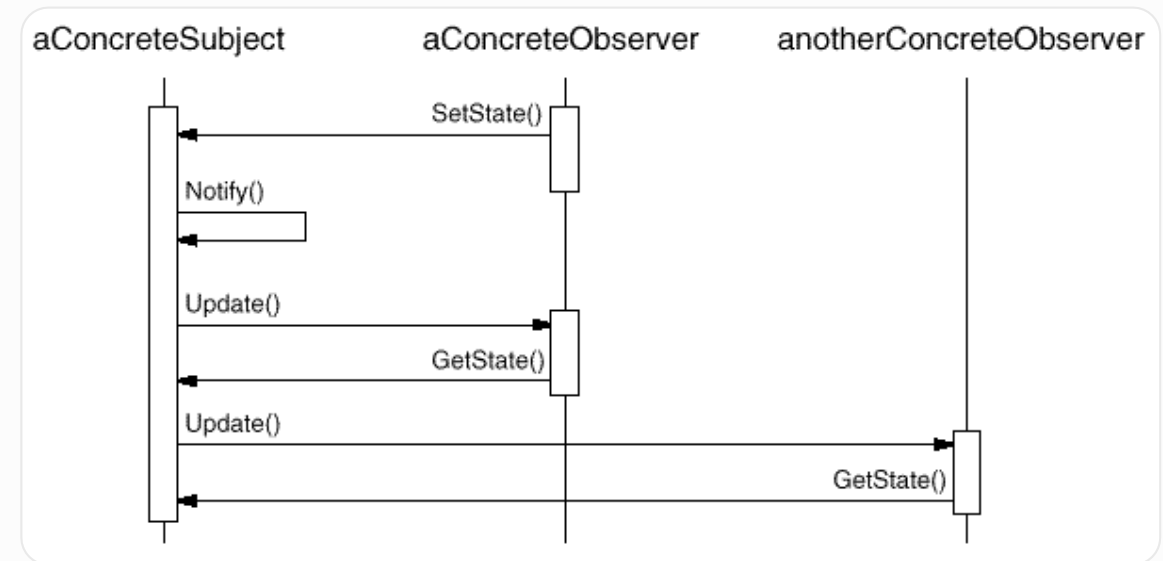


- Almacena el estado de interés para objetos **ObservadorConcreto**.
- Envía una notificación a sus observadores cuando su estado cambia

- Mantiene referencia al objeto **SujetoConcreto**
- Almacena el estado que debe ser consistente con el sujeto
- Implementa el método abstracto `actualizar`

Patrón de Proyecto Observador

- Formas de implementación:
 - Los propios observadores pueden enviar actualizaciones de estado al sujeto concreto, que a su vez notifica a todos los observadores
 - Los demás observadores pueden obtener el estado para saber si necesitan actualizarse o no
 - Otra forma de implementación sería que los observadores, al suscribirse, especifiquen algunas condiciones para que solo reciban la notificación si las condiciones son verdaderas
 - En el libro de Gamma hay varias otras consideraciones importantes a tener en cuenta al implementar este patrón



** Imagen capturada en el libro versión
html publicado en CD (Gamma et al 1995)*

Patrón de Proyecto Observador

- **Ejemplo:**
- diseñar la interfaz gráfica de usuario para un sistema de citas asignadas a los diversos veterinarios que trabajan en una clínica.
- En la vista de la secretaria, deben aparecer todas las citas
- En la vista de cada veterinario, aparece solo su agenda particular

Clínica Veterinaria X
Agenda del día 22/04/2006

8:00	Mariana/Lulu (Dr. João Carlos)
	Carlos/Beethoven (Dra. Maria Helena)
9:00	_____

10:00	Jessica/Tobias (Dra. Maria Helena)

Clínica Veterinaria X
Agenda del día 22/04/2006
Dr. João Carlos

8:00	Mariana/Lulu
9:00	_____

10:00	_____

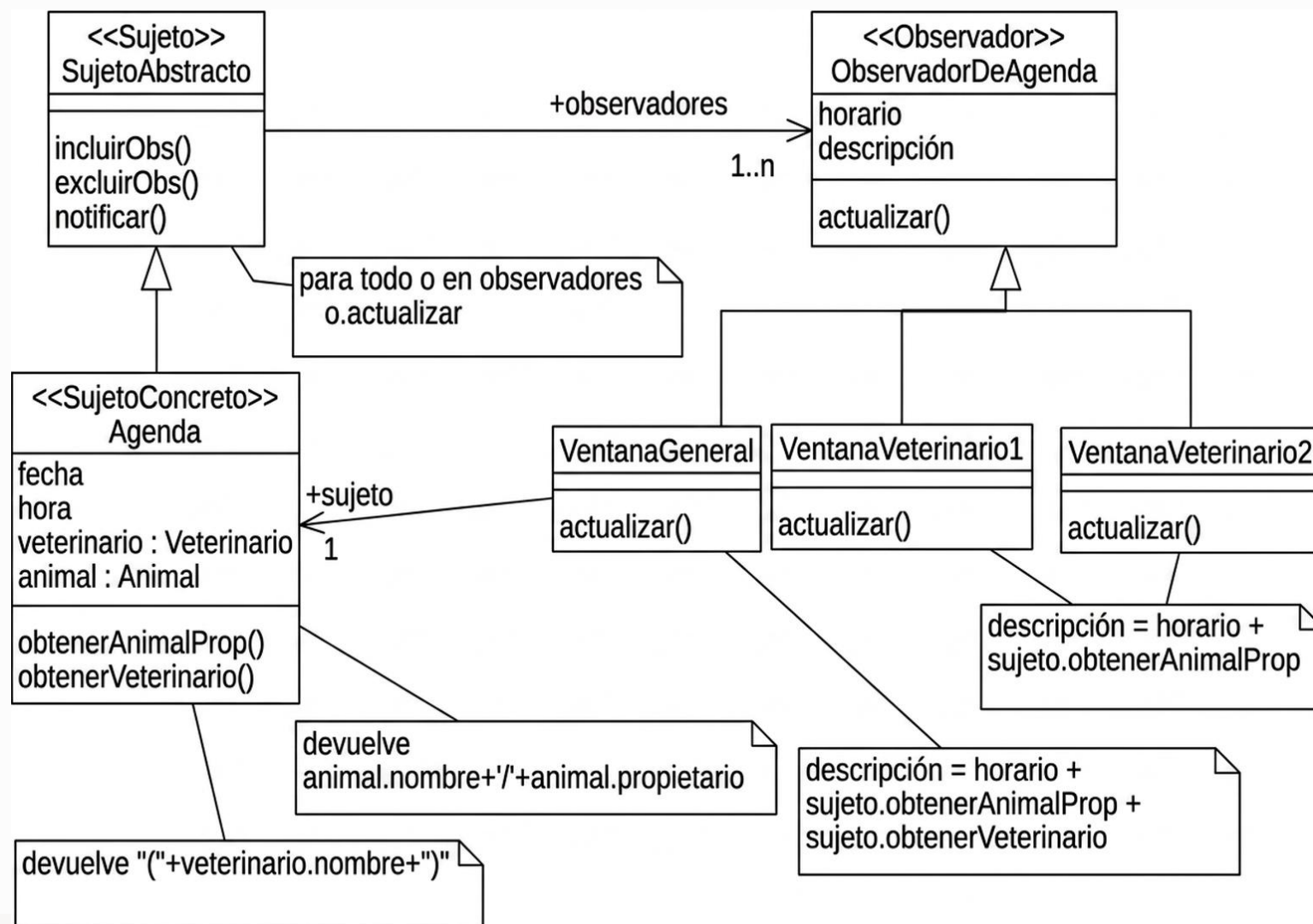
11:00	Fátima/Rex

Clínica Veterinaria X
Agenda del día 22/04/2006
Dra. Maria Helena

8:00	Carlos/Beethoven
9:00	_____

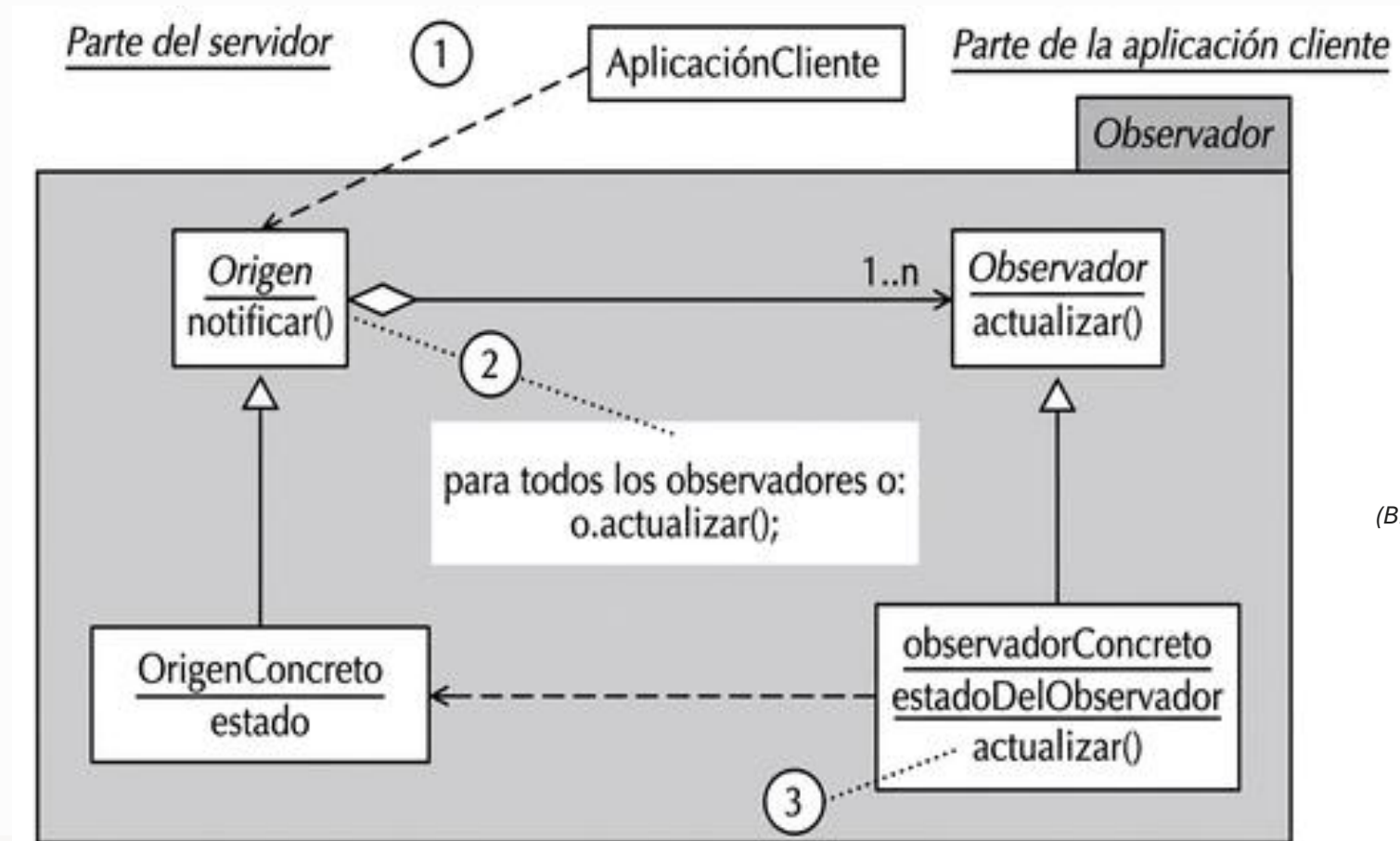
10:00	Jessica/Tobias
11:00	_____

Patrón de Proyecto Observador



Patrón de Proyecto Observador

- ¿Cómo funciona el Observador?

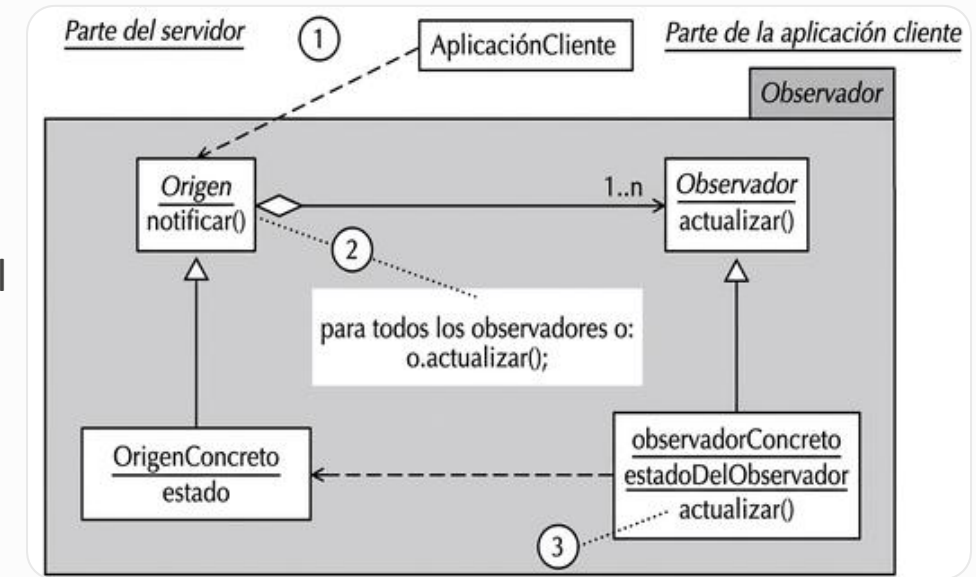


(Braude, 2005)

Patrón de Proyecto Observador

- ¿Cómo funciona?

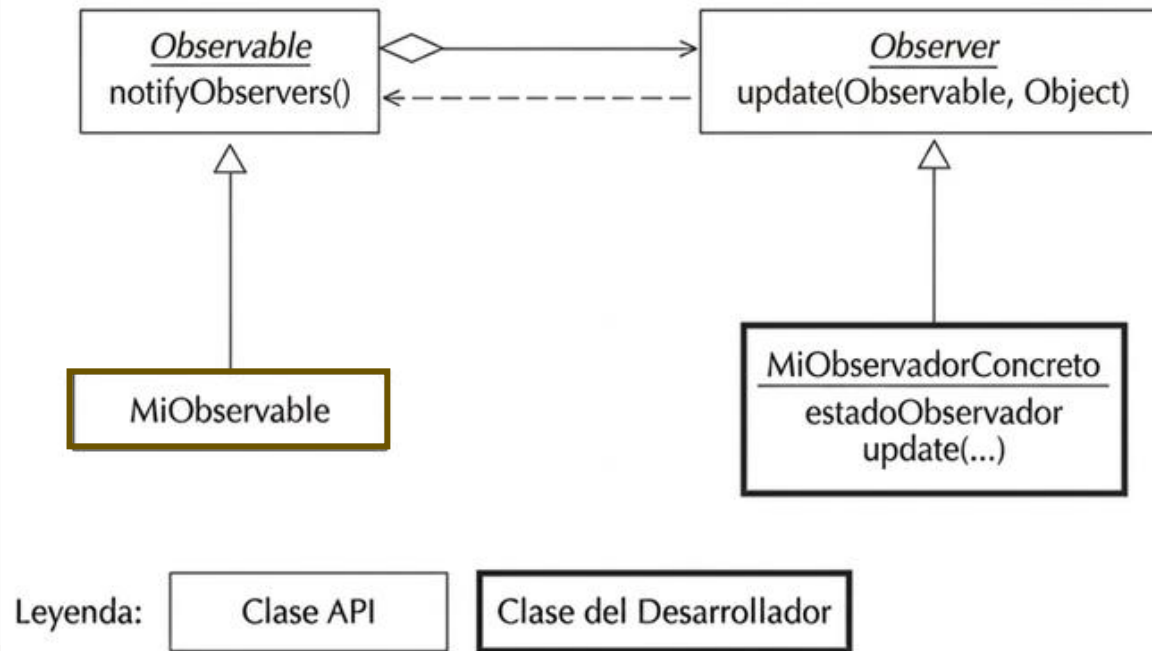
- (Paso 1) La aplicación cliente referencia un objeto de interfaz conocido, solicitando que los observadores sean notificados.
 - Por ejemplo, la aplicación cliente podría ser un proceso programado para alertar sobre una alteración de datos. En el modelo, esto se muestra como un objeto AplicacionCliente, que informa al objeto Origen para ejecutar su función notificar().
- (Paso 2) El método notificar() llama a la función actualizar() en cada objeto Observador que agrega.
- (Paso 3) La implementación de actualizar() depende del ObservadorConcreto particular al que pertenece.
 - Normalmente, actualizar() compara el estado del objeto ObservadorConcreto (valores de variables) con el de la fuente de datos central, para luego decidir si debe o no alterar sus valores de variables de la misma forma.



(Braude, 2005)

Patrón de Proyecto Observador

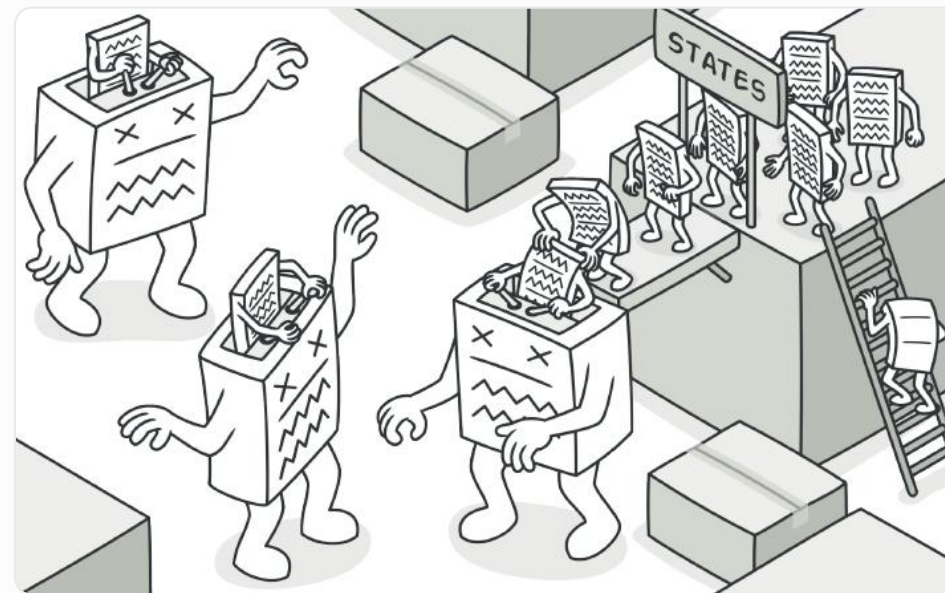
- Cabe recordar que este patrón existe en la API Java
 - Observe que *update()* es un método de devolución de llamada (*call-back*), porque proporciona a los objetos Observer una referencia a su origen, permitiendo que comparen sus datos, por ejemplo con el objeto Observable al ejecutar el *update*.
 - Como la actualización se realiza de modo retroactivo, no hay necesidad de que las clases concretas Observer mantengan referencias al objeto Observable.



(Braude, 2005)

Patrón de Proyecto Estado (del inglés *State*)

- **Clasificación:** Patrón de comportamiento/Ámbito de objetos
- **Intención:** Permitir que un objeto cambie su comportamiento cuando su estado interno cambie. La idea es hacer que parezca que el objeto ha cambiado de clase.
- **Aplicabilidad:**
 - cuando el comportamiento de un objeto depende de su estado, que puede cambiar en tiempo de ejecución
 - cuando este cambio de comportamiento implementado con el uso de estructuras *case* favorece errores, principalmente cuando el sistema evoluciona y surgen nuevos estados, ya que varias operaciones contendrán estas estructuras *case*.

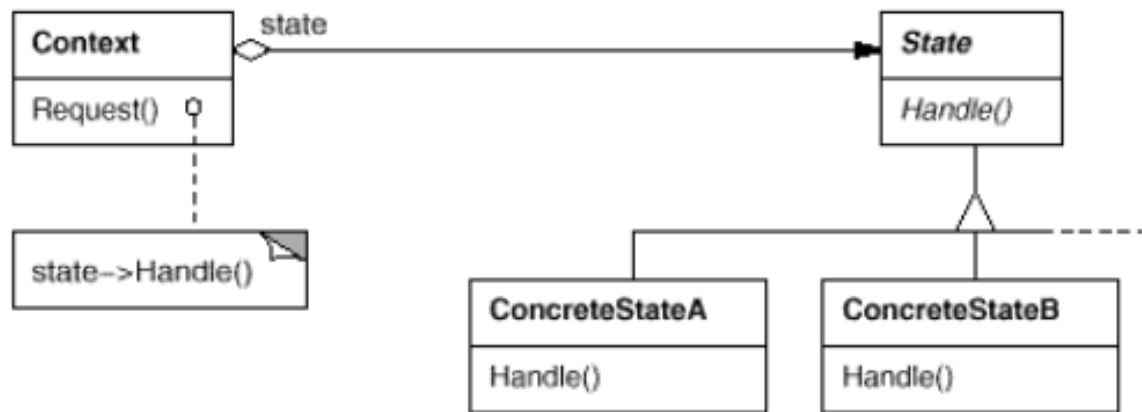


(Shvets, Alexander, 2018)

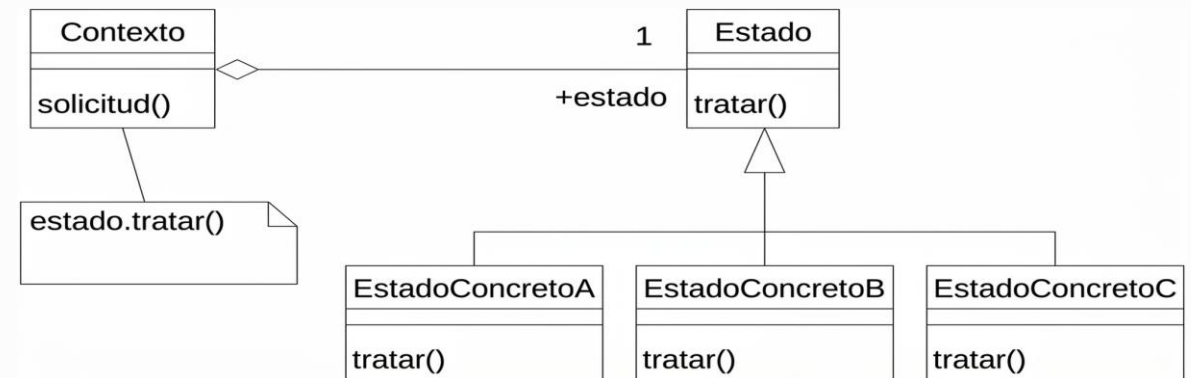
Patrón de Proyecto Estado

- **Estructura:**

- Crear una clase para representar el objeto en sí y otra para representar su estado
- El estado puede variar independientemente de otros objetos.



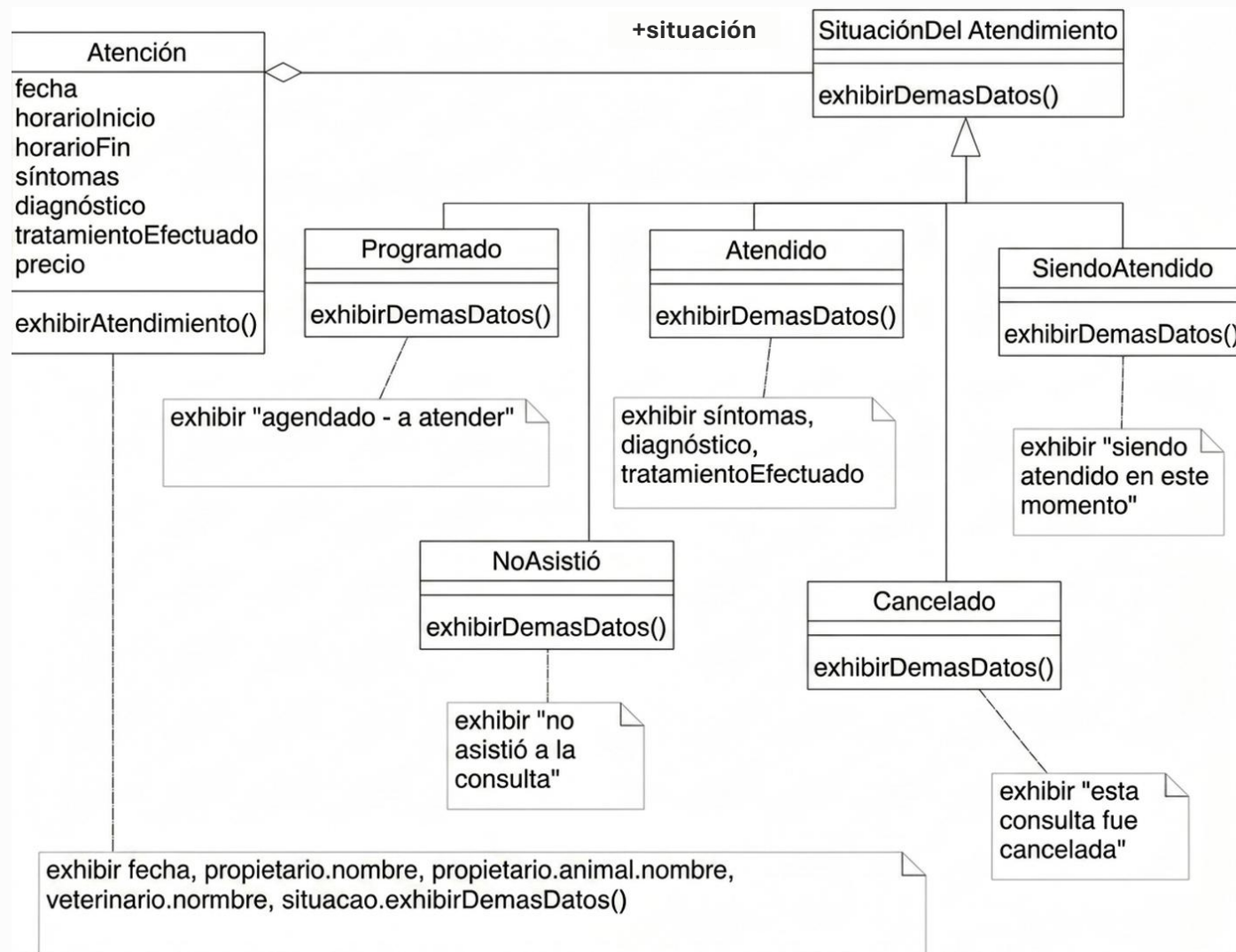
** Imagen capturada en el libro versión
html publicado en CD (Gamma et al 1995)



Patrón de Proyecto Estado

- **Ejemplo:**

- Mostrar un mensaje que tiene una parte fija y otra parte que depende del estado de la atención del paciente



Encuesta 10:
Uso del patrón
Estado



Patrón de Proyecto Estado

- Cuestiones de implementación:
 - ¿Quién define las transiciones de estado: contexto o estado?
 - Si hay un criterio fijo para el cambio de estado: contexto
 - Cambios constantes: estado
 - Interfaz necesaria para que sea posible que los objetos estado modifiquen explícitamente el estado actual del contexto
 - Tabla para mapear transiciones
 - Crear y destruir objetos Estado
 - ¿Crear todos los objetos a priori o crear/destruir bajo demanda?

Patrón de Proyecto Estado

- Ejemplo de uso de *State* en un juego:
 - *State · Design Patterns Revisited · Game Programming Patterns*

Patrón de Proyecto Estrategia (del inglés *Strategy*)

- **Clasificación:** Patrón de comportamiento/Ámbito de objetos
- **Intención:** Definir una familia de algoritmos, encapsular cada uno de los algoritmos y hacerlos intercambiables. La estrategia permite que el algoritmo varíe independientemente de los clientes que lo usan.
- **Aplicabilidad:**
 - varias clases relacionadas difieren solo en el comportamiento o
 - se necesitan diversas versiones de un algoritmo o
 - las aplicaciones cliente no necesitan conocer detalles específicos de las estructuras de datos de cada algoritmo



(Shvets, Alexander, 2018)

Patrón de Proyecto Estrategia

- Ejemplo:

- Crear una clase abstracta para la Estrategia empleada por el algoritmo, así como subclasses que especialicen cada uno de los algoritmos.
- El Contexto mantiene una referencia al objeto Estrategia y puede definir una interfaz para permitir que la Estrategia accese a sus datos. La Estrategia define una interfaz común a todos los algoritmos disponibles. El Contexto delega las solicitudes recibidas de las aplicaciones cliente a su estrategia.

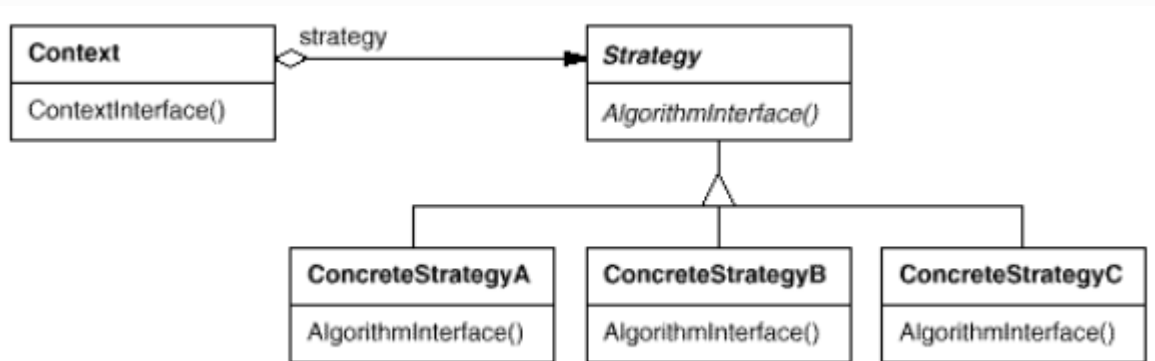
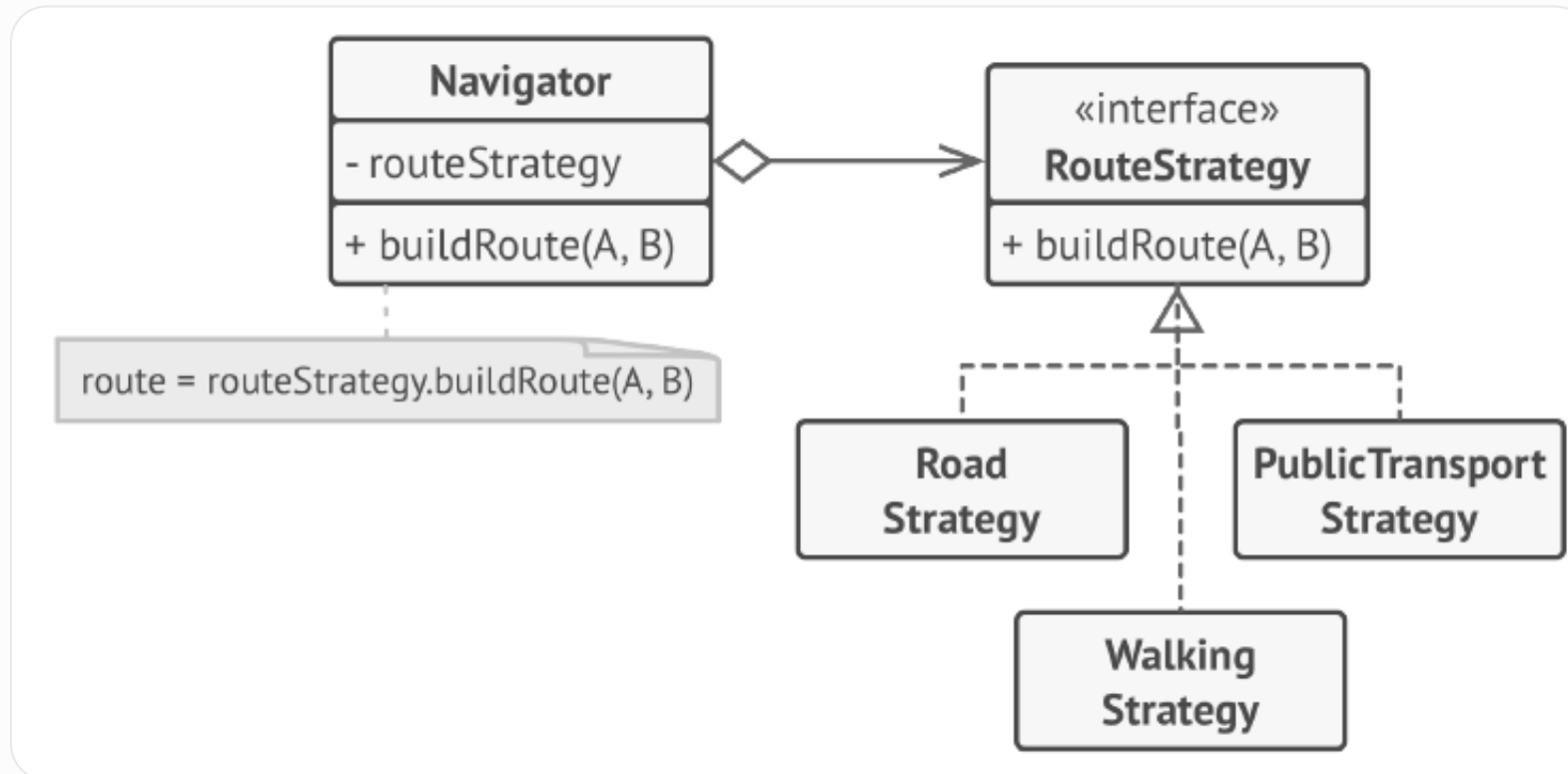


Imagen capturada del libro versión htm
publicado en CD (Gamma et al 1995)



Patrón de Proyecto Estrategia

- **Ejemplo:** El algoritmo para el cálculo de ruta depende de lo que el usuario elija



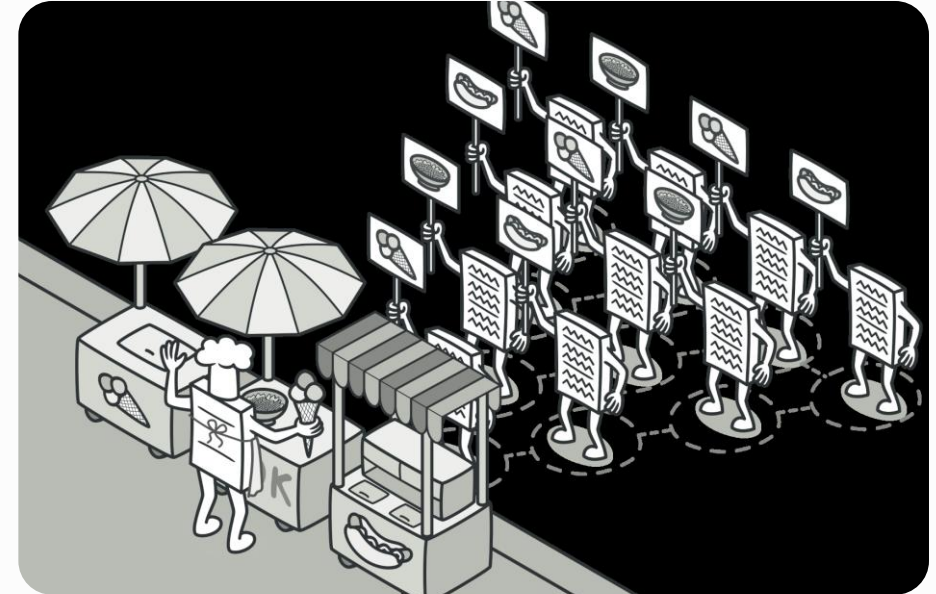
(Shvets, Alexander, 2018)

¿Diferencia entre *State* y *Strategy*?

- *State* puede considerarse una extensión de *Strategy*.
- Ambos patrones se basan en la composición: cambian el comportamiento del contexto delegando parte del trabajo a objetos auxiliares.
- *Strategy* hace que los objetos sean completamente independientes e inconscientes de los demás.
- Sin embargo, *State* no restringe las dependencias entre estados concretos, permitiendo que cambien el estado del contexto a voluntad.

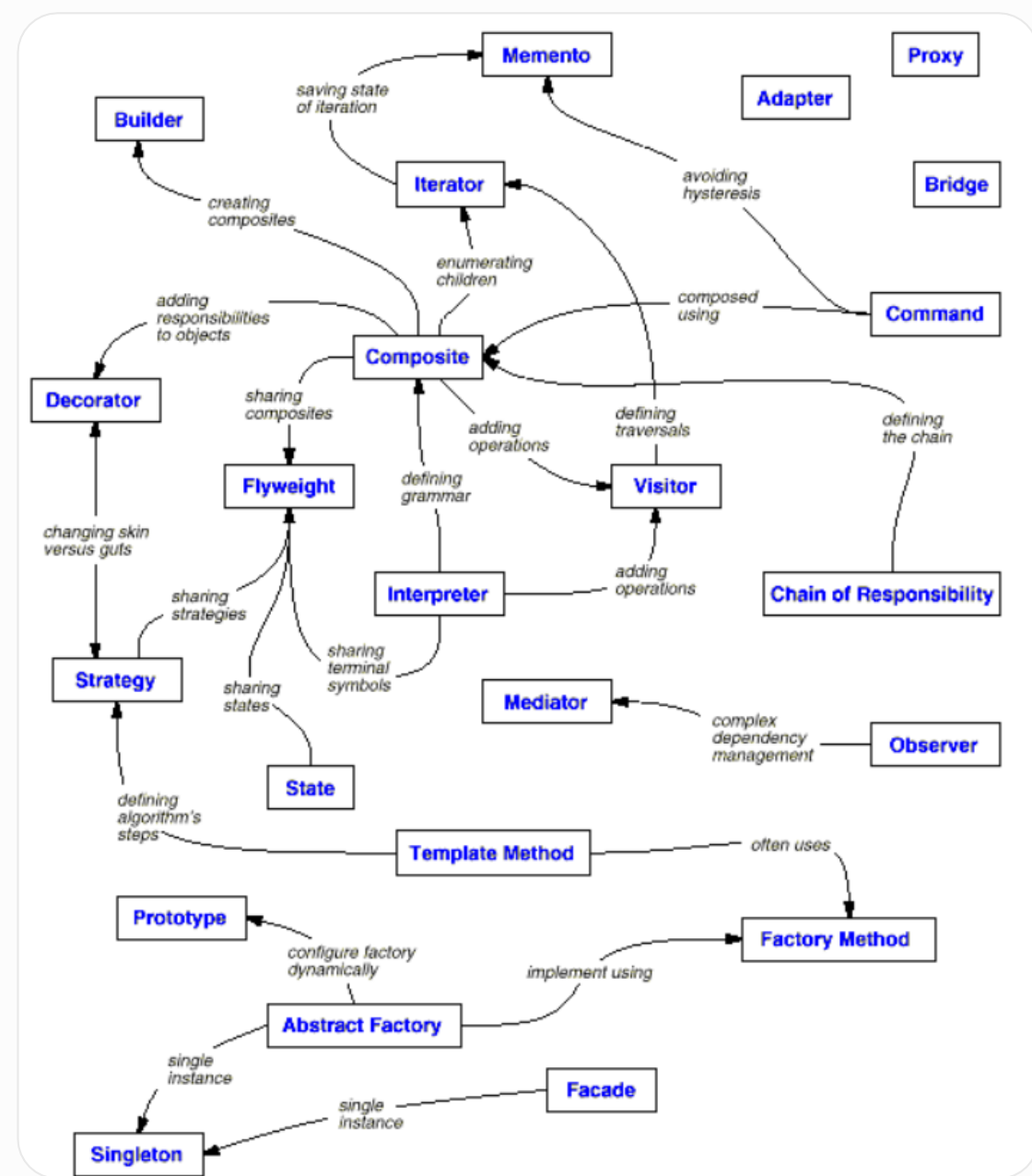
Patrón de Proyecto Visitante (del inglés *Visitor*)

- **Clasificación:** Patrón de comportamiento/Ámbito de objetos
- **Intención:** Permite que se ejecute una operación a lo largo de un conjunto con objetos de diferentes clases, pero teniendo un objeto “visitante” que implementa variantes de una misma operación, que corresponden a todas las clases objetivo.
 - Usa este patrón también cuando un comportamiento tiene sentido solo en algunas clases de la jerarquía, pero no en otras



(Shvets, Alexander, 2018)

Relações entre los Patrones de Proyecto



Conclusiones

Ventajas:

- Soluciones utilizadas en diversos proyectos y que han demostrado ser útiles
- Soluciones con ventajas y desventajas conocidas
- Ofrecen un vocabulario común entre desarrolladores
- Se adhieren a las mejores prácticas de desarrollo OO
- Sugieren sistemas extensibles, flexibles y robustos

Desventajas:

- Creación de diversas clases e interfaces auxiliares, generando más complejidad
- Pueden generar sobrecarga de memoria y procesamiento
- Pueden ser peores que soluciones simples cuando se utilizan en contextos equivocados

Discusión: ¿cuál es el impacto de la Inteligencia Artificial en el desarrollo de *software* usando patrones de proyecto?

- Algunos impactos visibles:
 - Nuevos patrones de proyecto específicos de IA: multiagentes, encadenamiento de *prompts*;
 - Adaptación de los patrones de proyecto tradicionales: Por ejemplo:
 - 1) los modelos pesados de IA pueden requerir adaptaciones en patrones como *Factory* y *Builder*;
 - 2) la necesidad de agilidad de los agentes de IA para responder dinámicamente al cambio de condiciones puede requerir cambios en patrones como *Observer* y *Strategy*.
- Impacto de los patrones en la sostenibilidad (mayor consumo de energía)

Discussão: ¿cuál es el impacto de la Inteligencia Artificial en el desarrollo de *software* usando patrones de proyecto?

- Cambios en el ciclo de desarrollo
 - Ayuda en la adopción de patrones de proyecto: las herramientas de IA pueden incorporar automáticamente estos patrones en el código, mejorando las elecciones arquitectónicas;
 - nuevas posibilidades para automatizar el proceso de reconocimiento, sugerencia y adaptación de patrones de diseño de *software* de forma eficaz, lo que resulta en la mejora general del desarrollo;
 - Problemas de mantenimiento: el código generado por IA puede ser más difícil de entender/mantener
 - pueden ocurrir problemas serios si los patrones no son comprendidos por el desarrollador responsable del mantenimiento (aumento de la deuda técnica).

Discussão: ¿cuál es el impacto de la Inteligencia Artificial en el desarrollo de *software* usando patrones de proyecto?

- Rol del ingeniero de *software*
 - Deja de ser “codificador” para ser arquitecto y orquestador (importancia de una buena especificación del *software*);
 - El juicio humano es más crítico para garantizar que el código generado por la IA se adhiera a los principios de proyecto fundamentales, incluidos los patrones de proyecto;
 - Es fundamental que el ingeniero de *software* garantice que el código permanezca mantenible (reduciendo la deuda técnica).

Discussão: ¿cuál es el impacto de la Inteligencia Artificial en el desarrollo de *software* usando patrones de proyecto?

- Concluyendo: la IA puede ayudar a impulsar el uso de patrones, por basarse en usos conocidos para solucionar problemas reales;
- Desde el surgimiento de los patrones de *software*, se dijo que, en el futuro, nadie necesitaría recordarlos, ya que se habrían incorporado naturalmente al desarrollo.
- La IA puede ayudar en esto, pero el rol del ingeniero de *software* al juzgar si el patrón fue bien aplicado sigue siendo importante (al menos por ahora).

Referencias

- Biggerstaff, Ted J. ; Perlis, Alan J. **Software Reusability: Concepts and Models**. Acm Press Frontier Series, 1989
- Alexander et al. **A Pattern Language**. New York: Oxford University Press, 1977.
- Gamma, E.; Helm, R.; Johnson, R.; Vlissides, J. **Design Patterns - Elements of Reusable Object-Oriented Software**. Addison-Wesley, 1995
- Meszaros, G.; Doble, J. **A pattern language for pattern writing**, cap. 29 in J. Coplien; D. Schmidt. Pattern Languages of Program Design, Reading-MA, Addison-Wesley, p. 529–574, 1998.
- Buschmann F. et. al. **Pattern-oriented software architecture - a system of patterns**. Wiley, 1996.
- Coplien, J. **Advanced C programming styles and idioms**. Reading-MA: Addison-Wesley, 1992.
- Shvets, Alexander. **Dive into design patterns**, Refactoring.Guru, 2018, disponíble para descargar en: <https://www.goodreads.com/book/show/43125355-dive-into-design-patterns>
- Braude, Eric. **Projeto de Software: Da programação à arquitetura: Uma abordagem baseada em Java**, Bookman, 2005.

¡Gracias!

Profa. Dra. Rosana T. Vaccare Braga | www.linkedin.com/in/rosana-braga-13778912

MBA **USP**
ESALQ